Order Book Analyzer – Status Report 5

FINTECH 512 - Software Engineering for FinTech

Duke University, Team Hotel

Full team of five members for the group project (section 1)

Rebeca Hechtman
NetId: rh398

Mike Lin
NetId: hl561

Mory Shi
NetId: cs836

Loki Luo
NetId: lz294

Alan Song
NetId: ys501

Agenda

01. Project Ideation

02. GitLab Repository and Issues

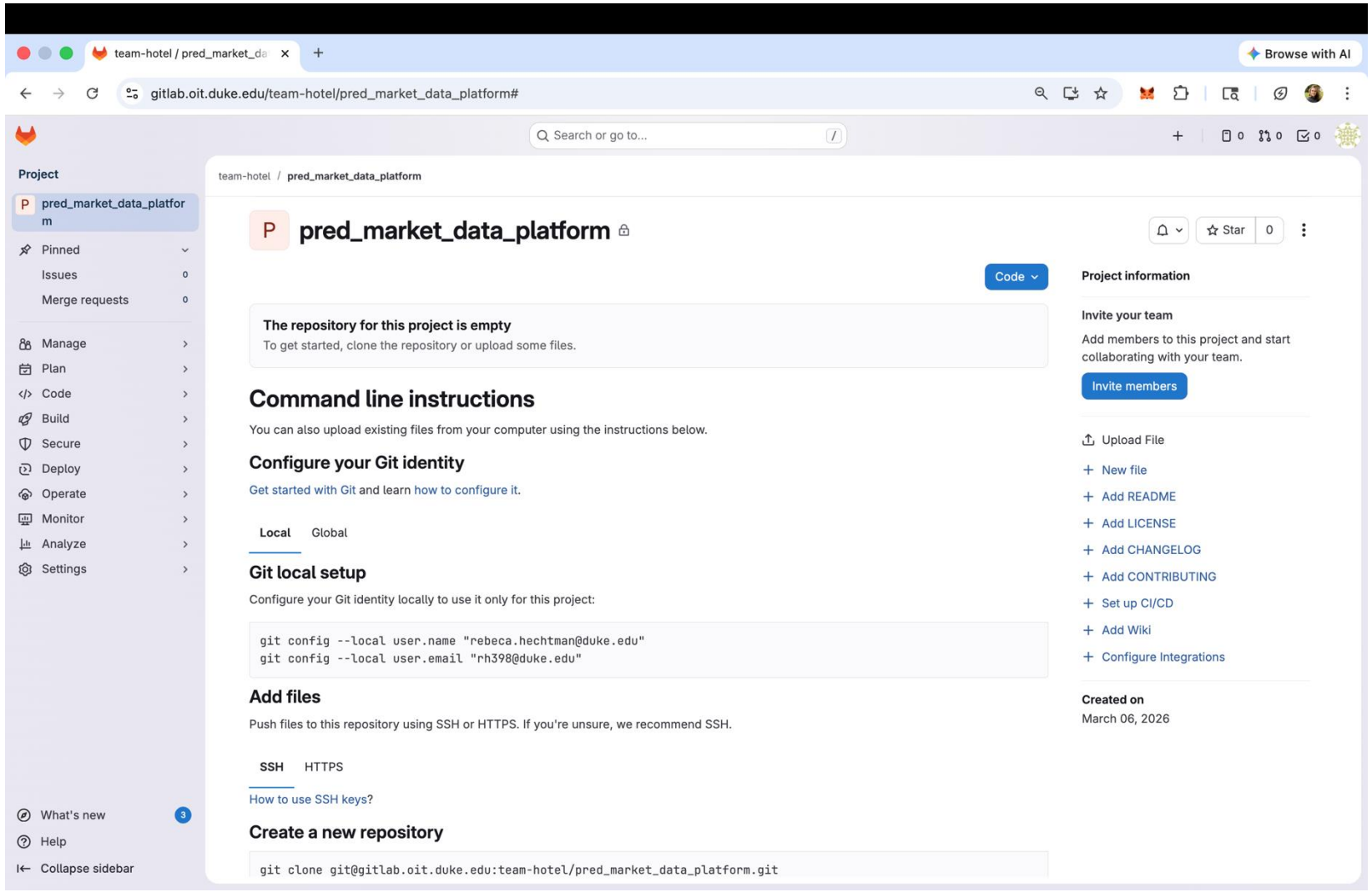
03. Updated Design

04. Status, Review and Planning

GitLab Repository and Issues

Team Project GitLab Repository

https://gitlab.oit.duke.edu/team-hotel/pred_market_data_platform#



Project Requirements – Organization and Ideation

Legend:
- Done
- In progress
- To be done

1. **Organization**
   1. W1: List the requirements
   2. W2: Refine the requirements
2. **Gitlab**
   1. W1: Create Gitlab
   2. W2: Update the issues (+ improve wording)
   3. W2: Understand Gitlab SAST Tooling
3. **System Architecture diagrams (C4)**
   1. W1: Create System Architecture
   2. W2: Create using C4 model
   3. W3 - Refine diagrams
4. **Virtual Machines (current version of site running on a Duke VCM)**
   1. W1: Apply VCM
   2. W5: Check how to share the VCM with the group
   3. W5: Share the VCM port with the group
   4. Deploy the app (Gitlab -> virtual machine)
   5. Automatically starting
5. **Security Analysis - DFD threat model**

   Create a Data Flow Diagram (DFD) for your BigBucks system, identify assets, actors, and threats, and perform a STRIDE analysis of DFD
   1. W2: 1st overview draft
   2. W3: Refinement and detailing
6. **Users and Administrative Functionality:**
   1. W1: initial user's type ideation
   2. W2: Admin's ideation
   3. W5: Detailing of user - Rebeca
   4. W5: Detailing of admin - Rebeca
7. **Data service**
   1. W1: data design ideation
   2. W1: data flow ideation (Polymarket API)
   3. W2: testing ideation
   4. W4: API to connect full stack with data service
8. **Full stack**
   1. W2-W3: Data design of user ideation
   2. W4: Data design of admin ideation - Alan
   3. W2: testing ideation
9. **Charting (3 or more + custom interaction)**
   1. W3: Ideation + paper prototype
   2. W5: Refine + Include custom interaction - Mory
   3. W5: Document with definitions - Mory
10. **Paper prototypes**
    1. W1 – W3: User Interface – Rebeca

    1- Home page for unauthenticated users + User authentication page; 2- New user registration page, 3- "Authenticated" home page for regular users, 4- Dashboard of relevant information, 5- User Information, 6- "Analysis" page that allows the user to make an informed decision to buy/sell a particular product, item, and or service + Allow the user to buy/sell a product, item, or service, 7- Analysis page of the user's current data, 8- History screen to show past transactions, 9- Reporting functionality
    2. W4-W5: Tabs for admin functionality: View all users, Disable user accounts, Reset user passwords, View 'system' transactions, Perform some sort of analysis across all accounts - Rebeca

Project Requirements – GitLab and Coding

Legend:
- Done
- In progress
- To be done

11. Organization
1. W2: Setup the overall project repo structure – Rebeca
2. W1-W6: Keep GitLab issues up-to-date - Alan
3. W3: Talk about merge requests - All

12. Full Stack
1. Front-end design
    1. W3-W4: CSS for styling
    2. W3-W4: HTML (display) + JavaScript (interaction) for all the page prototypes of user interface (for the pages that need data, use dummy data)
    3. W5: Connect with back-end API - Loki
    4. W5: Update with real data from API - Loki
    5. W5: HTML tabs for admin functionality (HTML + Java Script)- Loki
2. Back-end design:
    1. W4: JavaScript to connect front end with the API
    2. W5: Data base with user roles and permissions - Alan
    3. W4: OAuth2 Authentication Integration
3. Database design:
    1. W4: for user (PostgreSQL)
4. Tests suite (including DFD created)
    1. W5: Unit tests Front End – Loki
    2. W5: Unit tests Back End - Alan
5. Logging and Monitoring
    1. W5: Design - Alan
    2. W5: Implement - Alan

13. CI/CD – YAML file
1. Build release software on every commit
2. W5: Runs test suite on every commit - Mory
3. W5: Runs Gitlab SAST Tooling - Mory
4. Automatically deploy your software to your VCM

14. Data Service Design
1. W2: API to connect data service with Polymarket (Data Integration using a REST/JSON web API)
2. W4: API to connect full stack with data service
3. W2: Message queue
4. W2: Database (PostgreSQL) implementation
5. W4: Update transformation
6. W5: Resampling of transformation - Mike
7. W3-W5: Tests suite; also use DFD created – Mike

15. Containerize Application - All
1. W5: Create docker files to create an image of the application
2. W5: Create a docker compose file to deploy and execute image

16. Security Analysis – run tests against the software
1. Run Semgrep analyzer against your code base
2. Runs OWASP Zed Attack Proxy

Project Requirements – Detail Front End

Legend:
- Done
- In progress
- To be done

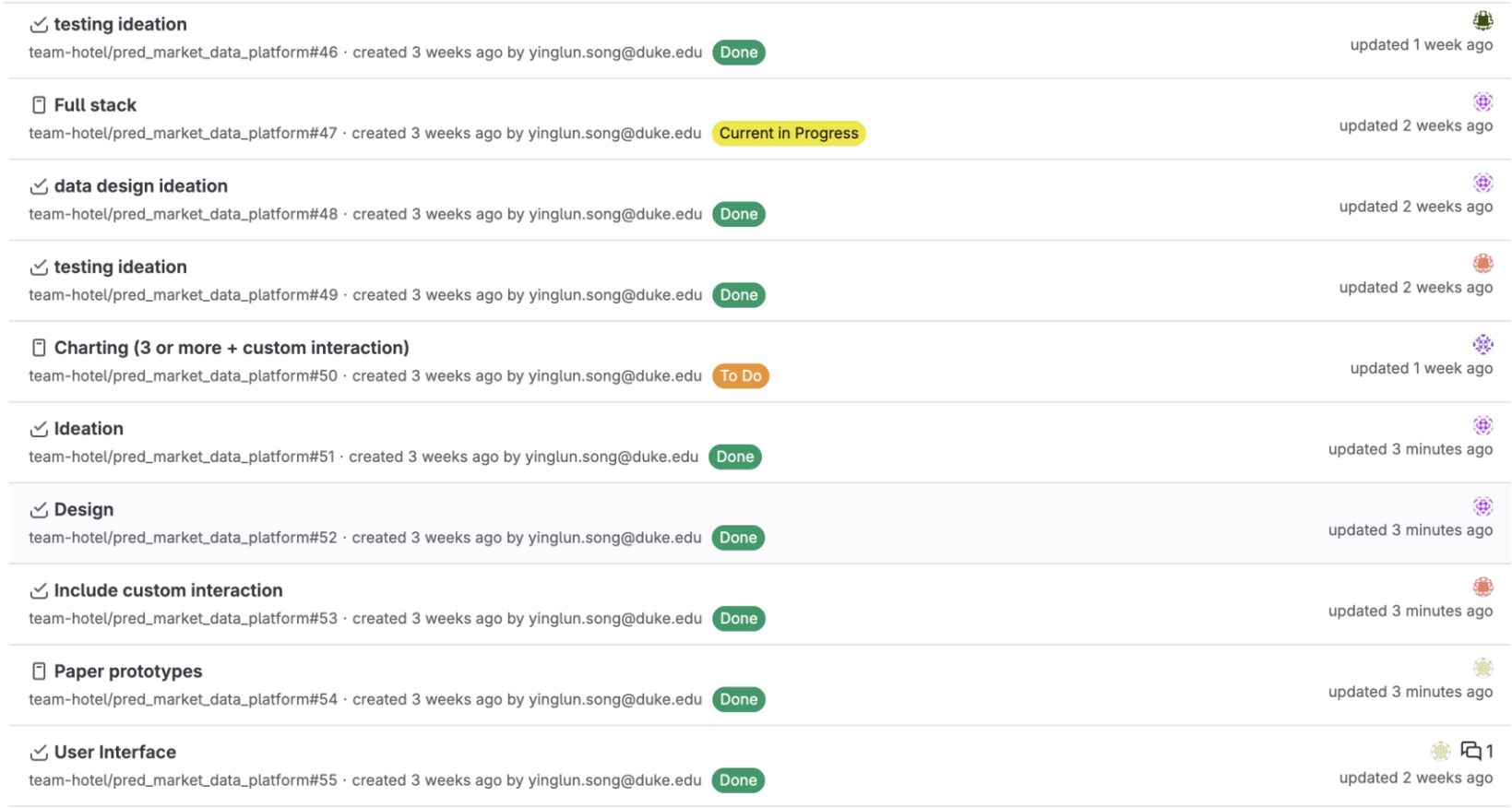
1. CSS for styling

2. HTML for all the page prototypes (for the pages that need data, using dummy data for now)
    1. Home page for unauthenticated users
    2. User authentication page
    3. New user registration page
    4. “Authenticated” home page for regular users. Dashboard of relevant information.
    5. “Analysis” page that allows the user to make an informed decision to buy/sell a particular product, item, and or service + Allow the user to buy/sell a product, item, or service
    6. Analysis page of the user's current data
    7. History screen to show past transactions
    8. Reporting functionality

3. HTML tabs for admin functionality:
    1. View all users, Disable user accounts
    2. Reset user passwords
    3. View 'system' transactions
    4. Perform some sort of analysis across all accounts

Project Issues
(updated on April 10)

Link to the issue page: https://gitlab.oit.duke.edu/team-hotel/pred_market_data_platform/-/issues?sort=created_asc&state=opened&last_page_size=20&page_before=eyJjcmVhdGVkX2F0IjoiMjAyNi0wMy0xOSAxMDozNTozMS41MDU4MzcwMDAgLTA0MDAiLCJpZCI6IjQ1NTQyIn0



*Some of the issues that are created

Project Ideation

Project's 'elevator pitch'

> ***Bloomberg of Prediction Markets***

Prediction markets, where people bet real money on the outcomes of future events (like whether Bitcoin will hit $110,000 this year), are exploding in popularity. But making sense of all that betting activity can be frustrating. The raw data behind these markets is messy, disorganized, and hard to easily analyze.

Order Book Analyzer cuts through that noise. Think of us as the Bloomberg for prediction markets. We take massive amounts of complicated, second-by-second betting data and translate it into clear, powerful dashboards and insights. Instead of wrestling with messy data, our platform gives you the exact trends and historical information you need to see exactly where the smart money is going.

Project Scope Disclaimer: This project will be focused and dedicated exclusively to Polymarket BTC 15-min Up/Down (Phase 1). Future market expansions remain under evaluation based on technical complexity and timeline constraints.

1. Access & Opacity: High-frequency data (L2/L3 order books) is notoriously difficult to access, painful data wrangling and gaps.

2. Capacity to Process: Normalizing raw prediction market streams into usable research formats requires specialized engineering effort.

3. Signal-to-Noise Ratio: Traditional financial data is too noisy for pure behavioral signal extraction

A website provides Insights & Dashboards:

Insights, dashboard, and KPI for one or more popular prediction markets trades (Phase 1: Polymarket's BTC 15-min Up/Down, Phase 2: Expand prediction asset classes)

High quality, tick-level historical data and analytics for high-frequency prediction markets, starting with ultra-short-term crypto volatility (15-minute Up/Down from Polymarket)

1. **Curated Historical Data Archive**: Because live data is fleeting and heavy to store, reconstructing this past data is nearly impossible for new entrants.
2. **Domain & Technical Expertise**

1. Mid and small size Hedge Funds
2. Sophisticated algo traders who wants to trade in prediction market

1. Eagle Alpha
2. Dune Analytics
3. DIY engineering
4. Basic Polymarket Dashboards

Growth:

1. # of users
2. Retention
3. API call volumes
4. Daily Active User

System Health:

1. Yahoo Finance for Prediction Market
2. Bloomberg for Prediction Market

1. Social Media (LinkedIn, X, Reddit, Red Note)
2. Organic Growth
3. Search Engine
4. Ads

Crypto-native quants and algo traders currently hitting the limitations of Polymarket's visual dashboards, specifically those running high-frequency arbitrage or momentum strategies on BTC short-term derivative markets.

Data Infrastructure: Heavy cloud compute and storage costs (AWS EC2/S3, hosting specialized streaming and database clusters) to maintain gapless WebSocket ingestion. Variable costs for data query and usage.

Engineering & People: Time and resources dedicated to pipeline maintenance, API integration, and handling schema changes from the source markets.

Customer Acquisition & Marketing: Sales outreach, conference attendance, and content creation.

Compliance: Legal frameworks and terms of service management.

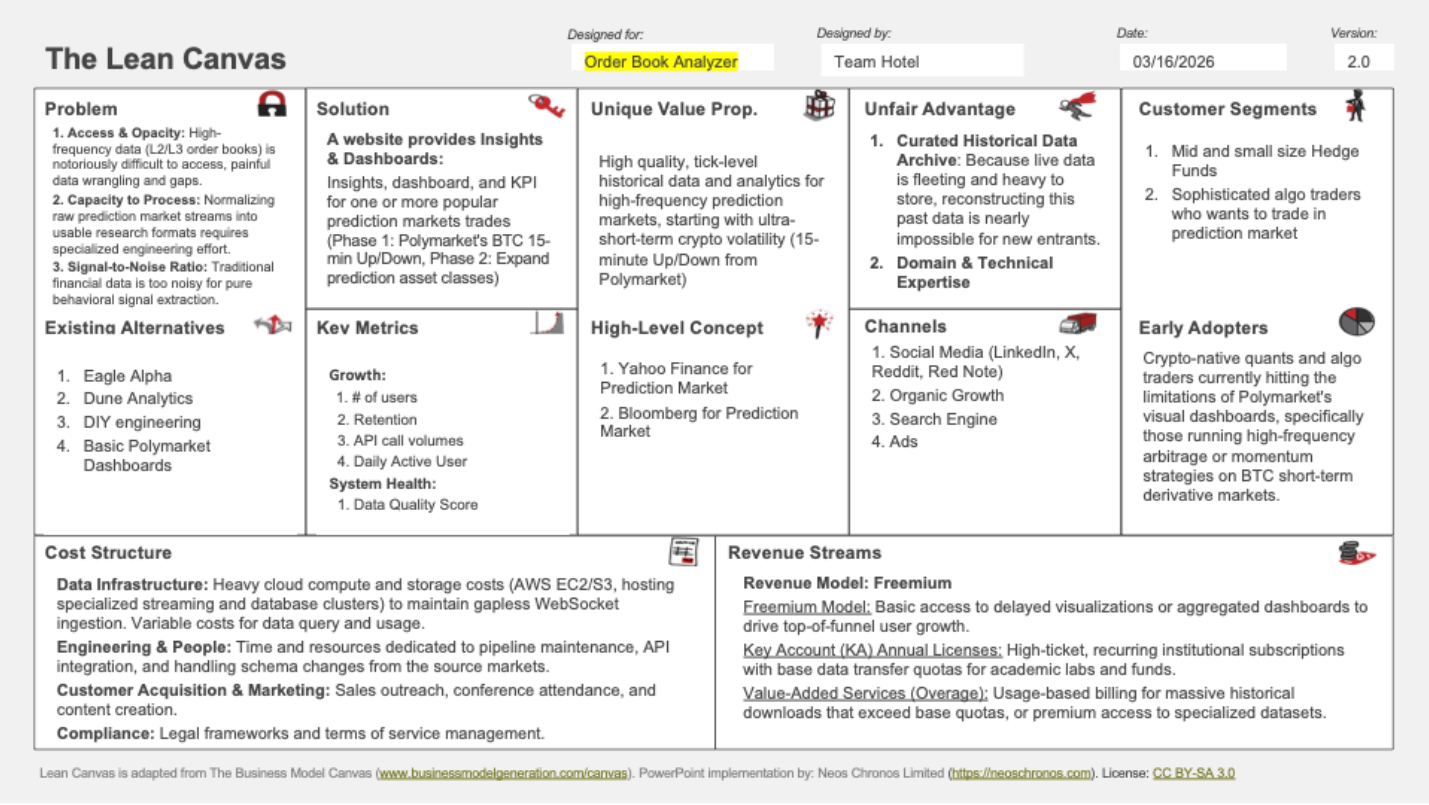
Revenue Model: Freemium

Freemium Model: Basic access to delayed visualizations or aggregated dashboards to drive top-of-funnel user growth.

Key Account (KA) Annual Licenses: High-ticket, recurring institutional subscriptions with base data transfer quotas for academic labs and funds.

Value-Added Services (Overage): Usage-based billing for massive historical downloads that exceed base quotas, or premium access to specialized datasets.

Changes we've made since the project started



The Lean Canvas

Designed for: Order Book Analyzer | Designed by: Team Hotel | Date: 03/16/2026 | Version: 2.0

Problem

1. **Access & Opacity:** High-frequency data (L2/L3 order books) is notoriously difficult to access, painful data wrangling and gaps.
2. **Capacity to Process:** Normalizing raw prediction market streams into usable research formats requires specialized engineering effort.
3. **Signal-to-Noise Ratio:** Traditional financial data is too noisy for pure behavioral signal extraction.

Existing Alternatives

1. Eagle Alpha
2. Dune Analytics
3. DIY engineering
4. Basic Polymarket Dashboards

Solution

A website provides Insights & Dashboards:

Insights, dashboard, and KPI for one or more popular prediction markets trades (Phase 1: Polymarket's BTC 15-min Up/Down, Phase 2: Expand prediction asset classes)

Key Metrics

Growth:

1. # of users
2. Retention
3. API call volumes
4. Daily Active User

System Health:

1. Data Quality Score

Unique Value Prop.

High quality, tick-level historical data and analytics for high-frequency prediction markets, starting with ultra-short-term crypto volatility (15-minute Up/Down from Polymarket)

High-Level Concept

1. Yahoo Finance for Prediction Market
2. Bloomberg for Prediction Market

Unfair Advantage

1. **Curated Historical Data Archive**: Because live data is fleeting and heavy to store, reconstructing this past data is nearly impossible for new entrants.
2. **Domain & Technical Expertise**

Channels

1. Social Media (LinkedIn, X, Reddit, Red Note)
2. Organic Growth
3. Search Engine
4. Ads

Customer Segments

1. Mid and small size Hedge Funds
2. Sophisticated algo traders who wants to trade in prediction market

Early Adopters

Crypto-native quants and algo traders currently hitting the limitations of Polymarket's visual dashboards, specifically those running high-frequency arbitrage or momentum strategies on BTC short-term derivative markets.

Cost Structure

Data Infrastructure: Heavy cloud compute and storage costs (AWS EC2/S3, hosting specialized streaming and database clusters) to maintain gapless WebSocket ingestion. Variable costs for data query and usage.

Engineering & People: Time and resources dedicated to pipeline maintenance, API integration, and handling schema changes from the source markets.

Customer Acquisition & Marketing: Sales outreach, conference attendance, and content creation.

Compliance: Legal frameworks and terms of service management.

Revenue Streams

Revenue Model: Freemium

Freemium Model: Basic access to delayed visualizations or aggregated dashboards to drive top-of-funnel user growth.

Key Account (KA) Annual Licenses: High-ticket, recurring institutional subscriptions with base data transfer quotas for academic labs and funds.

Value-Added Services (Overage): Usage-based billing for massive historical downloads that exceed base quotas, or premium access to specialized datasets.

Scope Restriction:

- This project will be focused and dedicated exclusively to Polymarket BTC 15-min Up/Down (Phase 1).
- Future market expansions remain under evaluation based on technical complexity and timeline constraints.

User and Admin Interface Prototype

Users Requirement: Role Segregation

Regular Users (Traders & Analysts)

These users interact directly with the prediction market platform to perform trading transactions and analysis

- Capabilities: Can register, authenticate, view personalized dashboards, analyze market data, and execute buy/sell transactions
- Subscription Tiers: "Normal" users are further segmented into Free, Pro, and Elite plans, which dictate their platform limits and volume capabilities
- Data Isolation: Normal users only have access to their own account details, transaction history, and analysis pages

Administrative Users

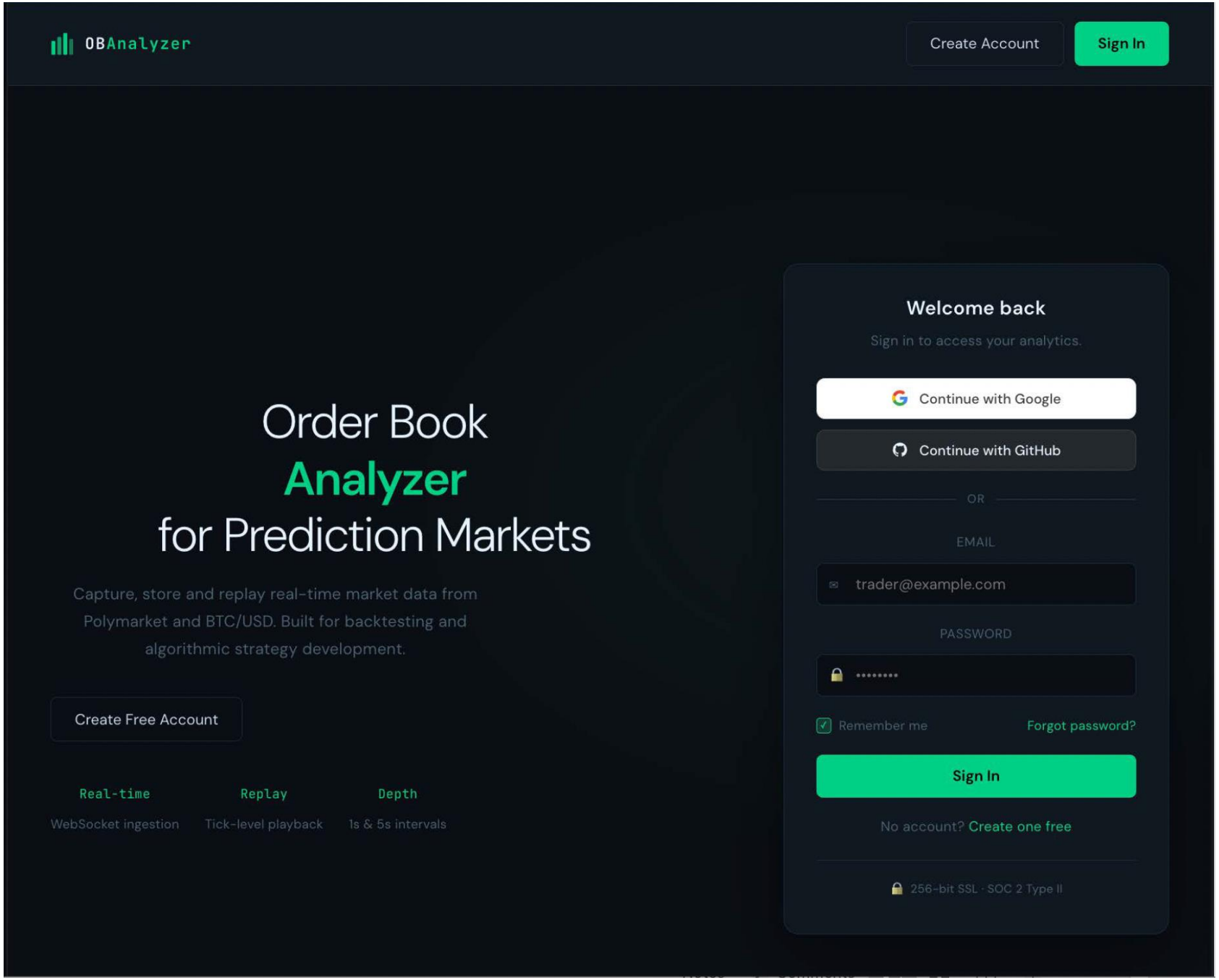
These users manage platform operations and oversee the user base via the dedicated "OBAnalyzer ADMIN" portal

- Capabilities: Can view the entire user directory, disable specific user accounts, and trigger password resets
- System Oversight: Have access to global platform analysis (e.g., MRR, churn rates) and system-wide transaction event logs

Prototype Screens

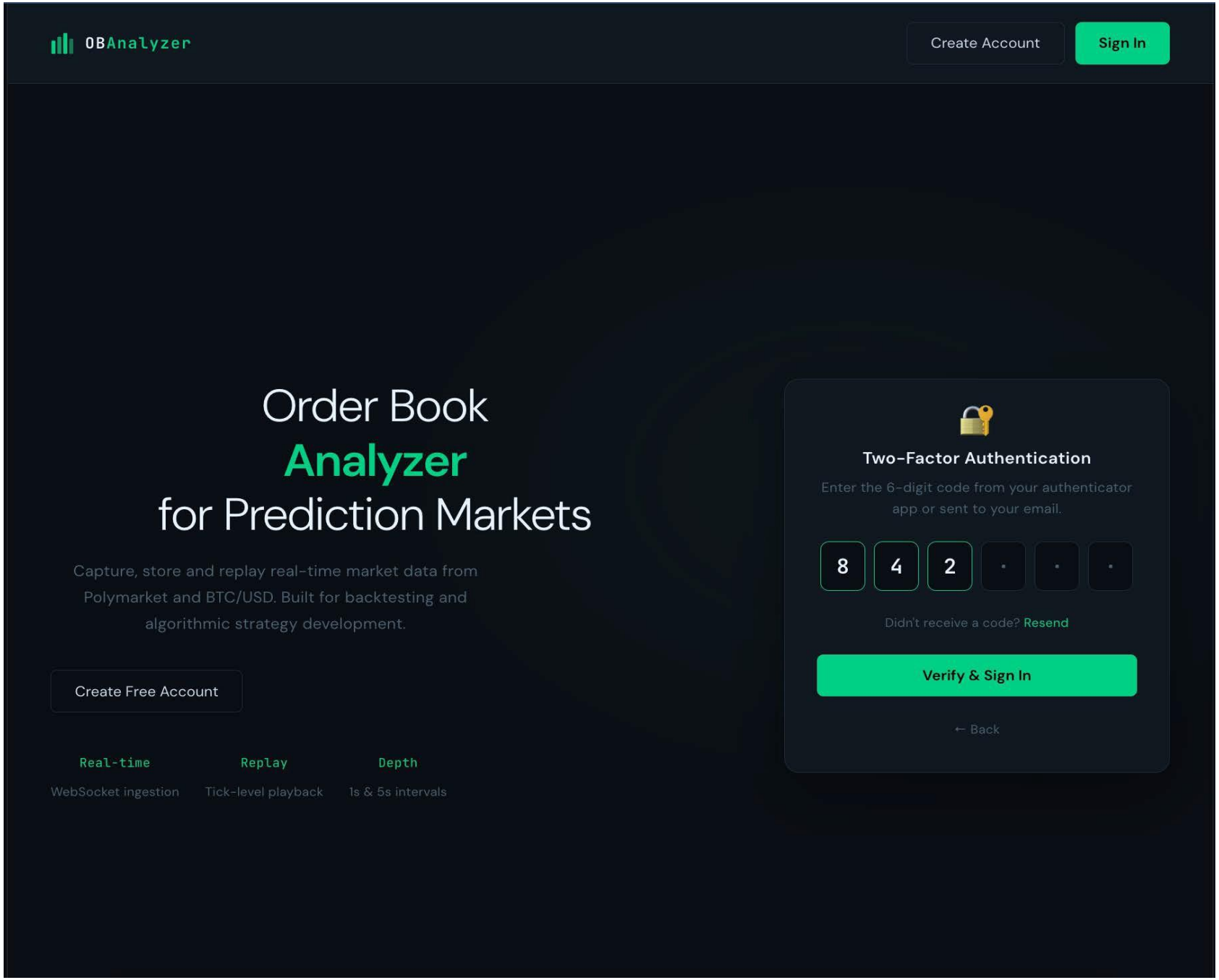
1- Home Page

for unauthenticated users + User authentication page



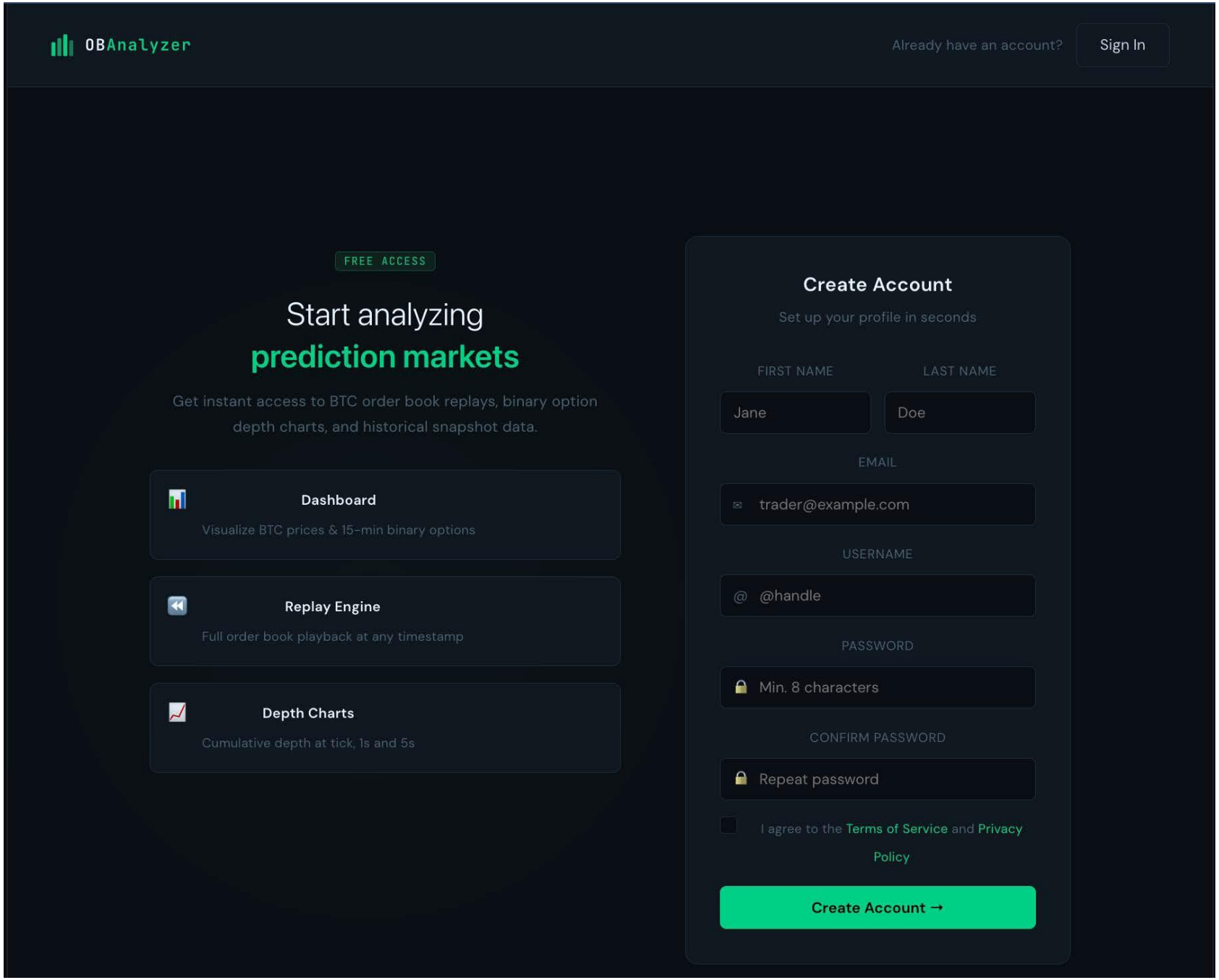
Prototype Screens

1b- User Authenti-cation



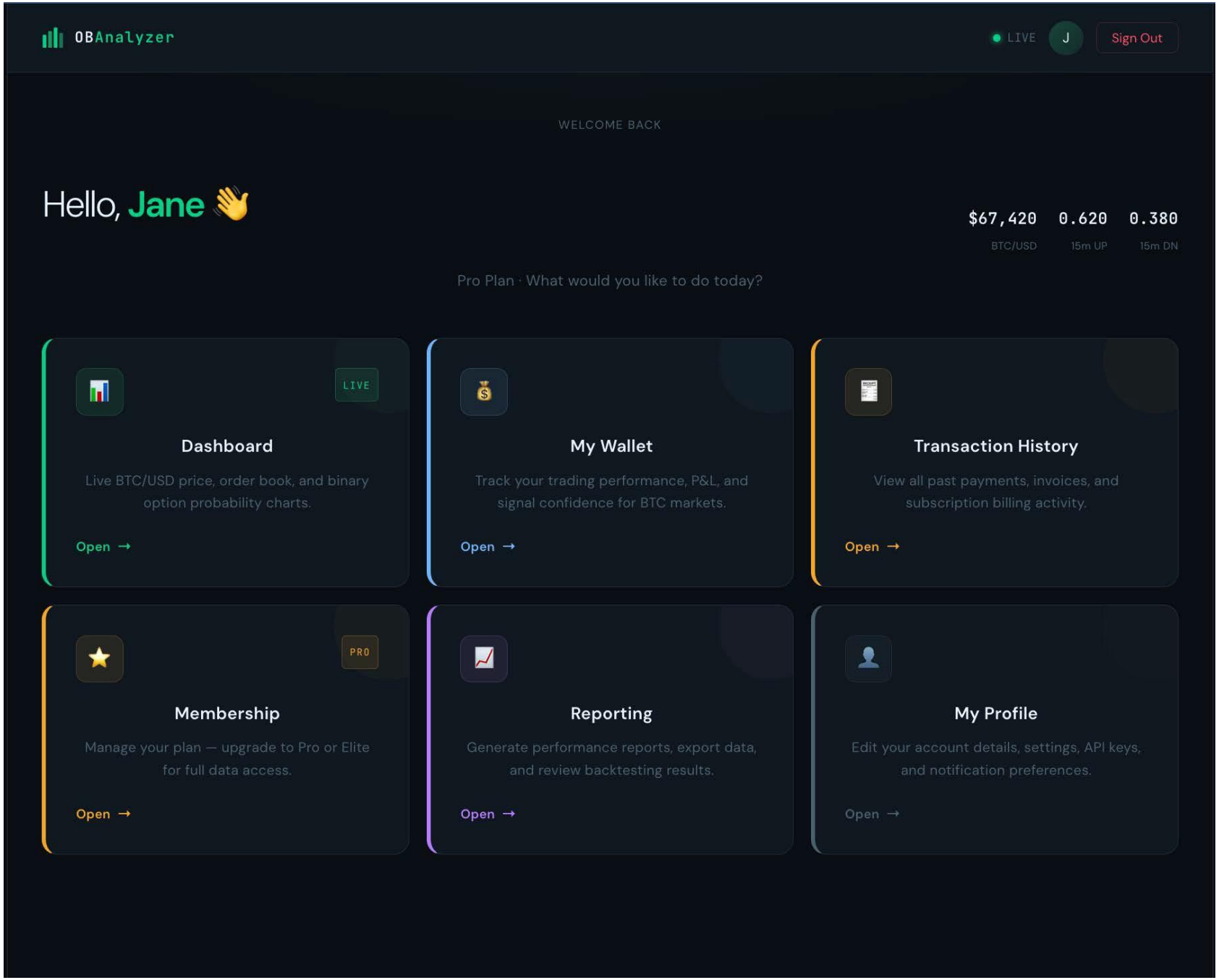
Prototype Screens

2- New user registration page



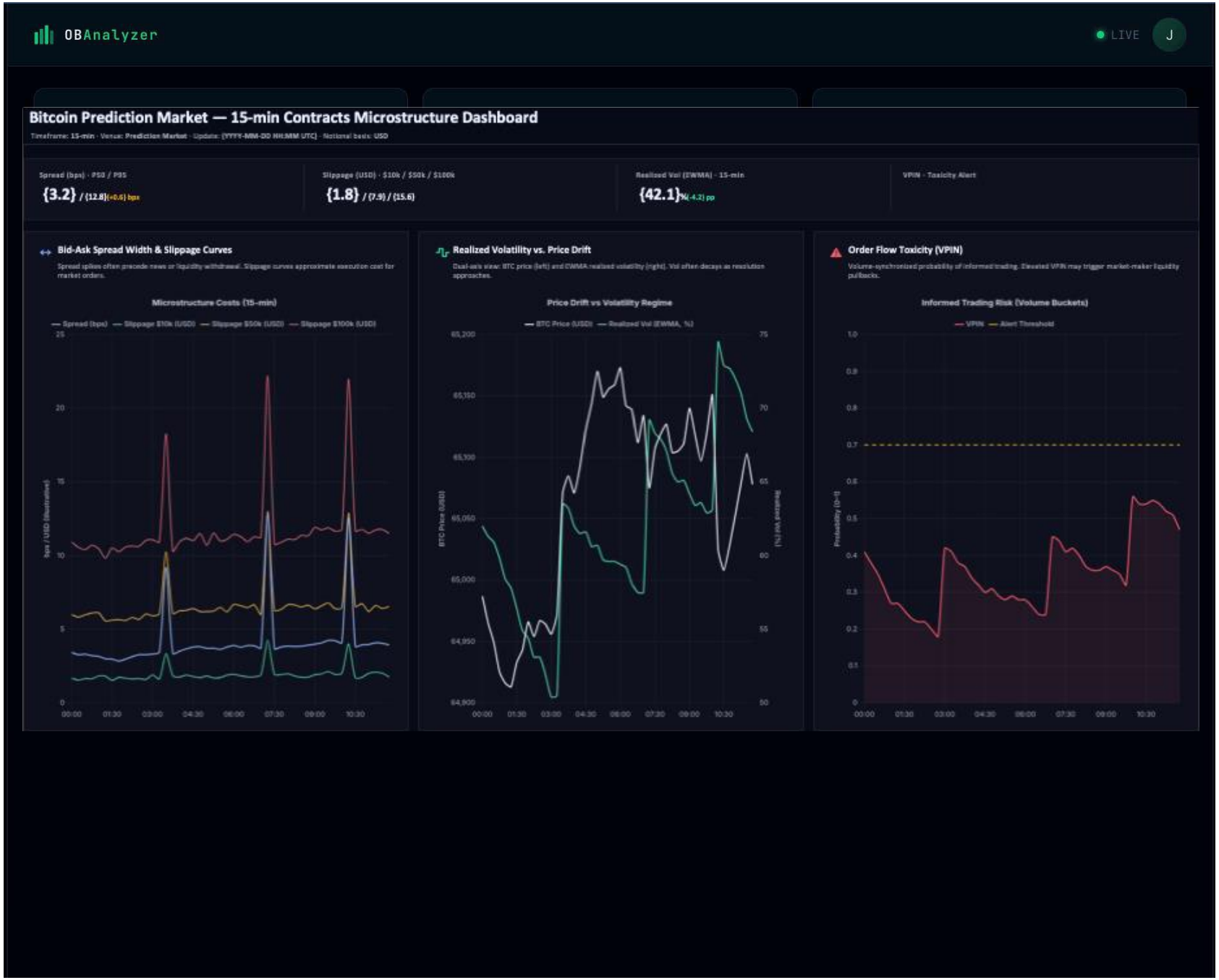
Prototype Screens

3- Authenti-cated home page for regular users



Prototype Screens

4- Dashboard of relevant information



Charting – v1 Ideation

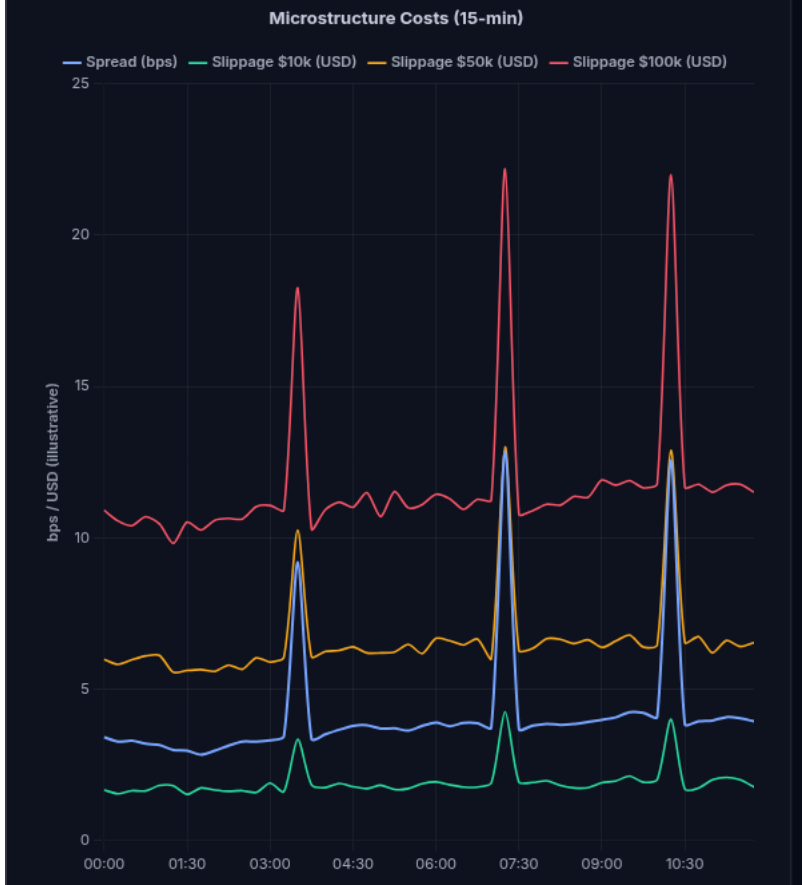
Bitcoin Prediction Market — 15-min Contracts Microstructure Dashboard

Timeframe: **15-min** · Venue: **Prediction Market** · Update: **{YYYY-MM-DD HH:MM UTC}** · Notional basis: **USD**

Spread (bps) · P50 / P95	Slippage (USD) · $10k / $50k / $100k	Realized Vol (EWMA) · 15-min	VPIN · Toxicity Alert
{3.2} / {12.8} {+0.6} bps	**{1.8}** / {7.9} / {15.6}	**{42.1}**% {-4.2} pp	**{0.62}** HIGH (threshold: 0.70)

↔ Bid-Ask Spread Width & Slippage Curves

Spread spikes often precede news or liquidity withdrawal. Slippage curves approximate execution cost for market orders.

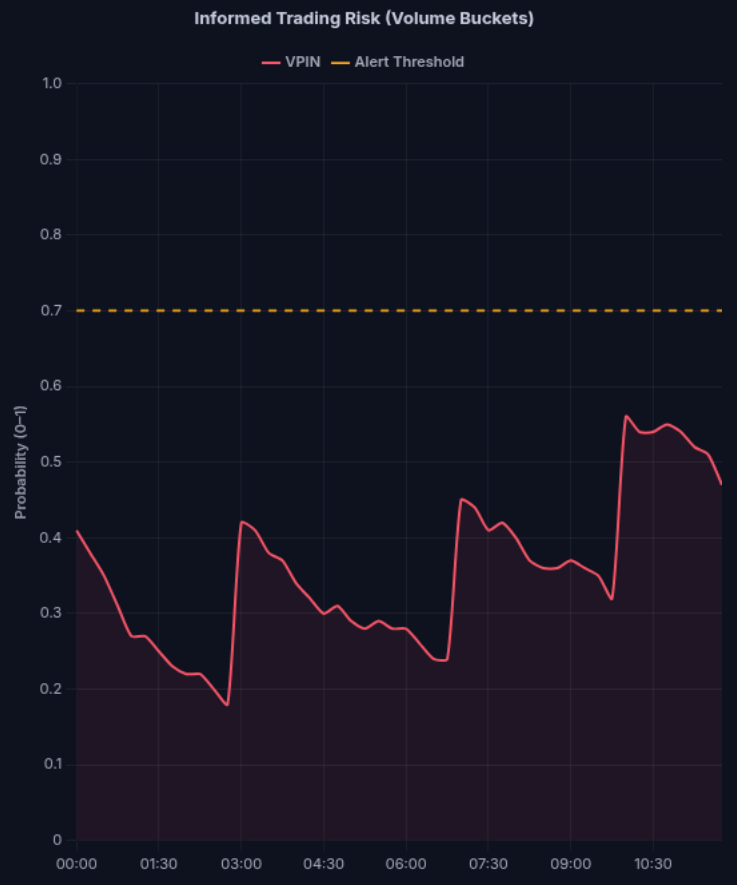


Realized Volatility vs. Price Drift

Dual-axis view: BTC price (left) and EWMA realized volatility (right). Vol often decays as resolution approaches.

⚠ Order Flow Toxicity (VPIN)

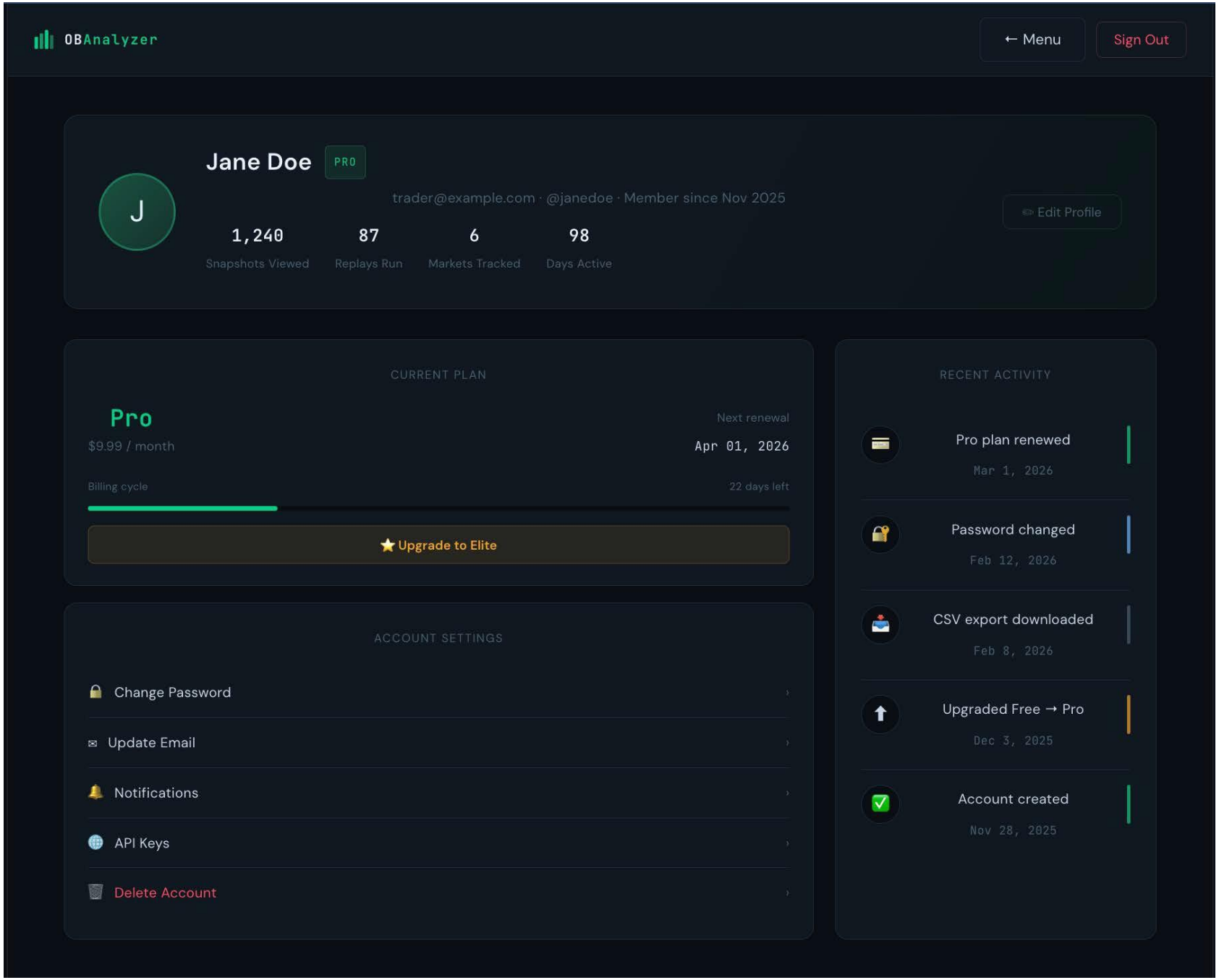
Volume-synchronized probability of informed trading. Elevated VPIN may trigger market-maker liquidity pullbacks.



Method notes: Spread = (Ask − Bid)/Mid (bps). Slippage curves are theoretical cost estimates (illustrative) calibrated to instantaneous depth. Realized vol shown as EWMA of 1-min returns annualized (placeholder). VPIN computed on volume buckets (placeholder). Replace placeholders with your venue's tick/orderbook feed.

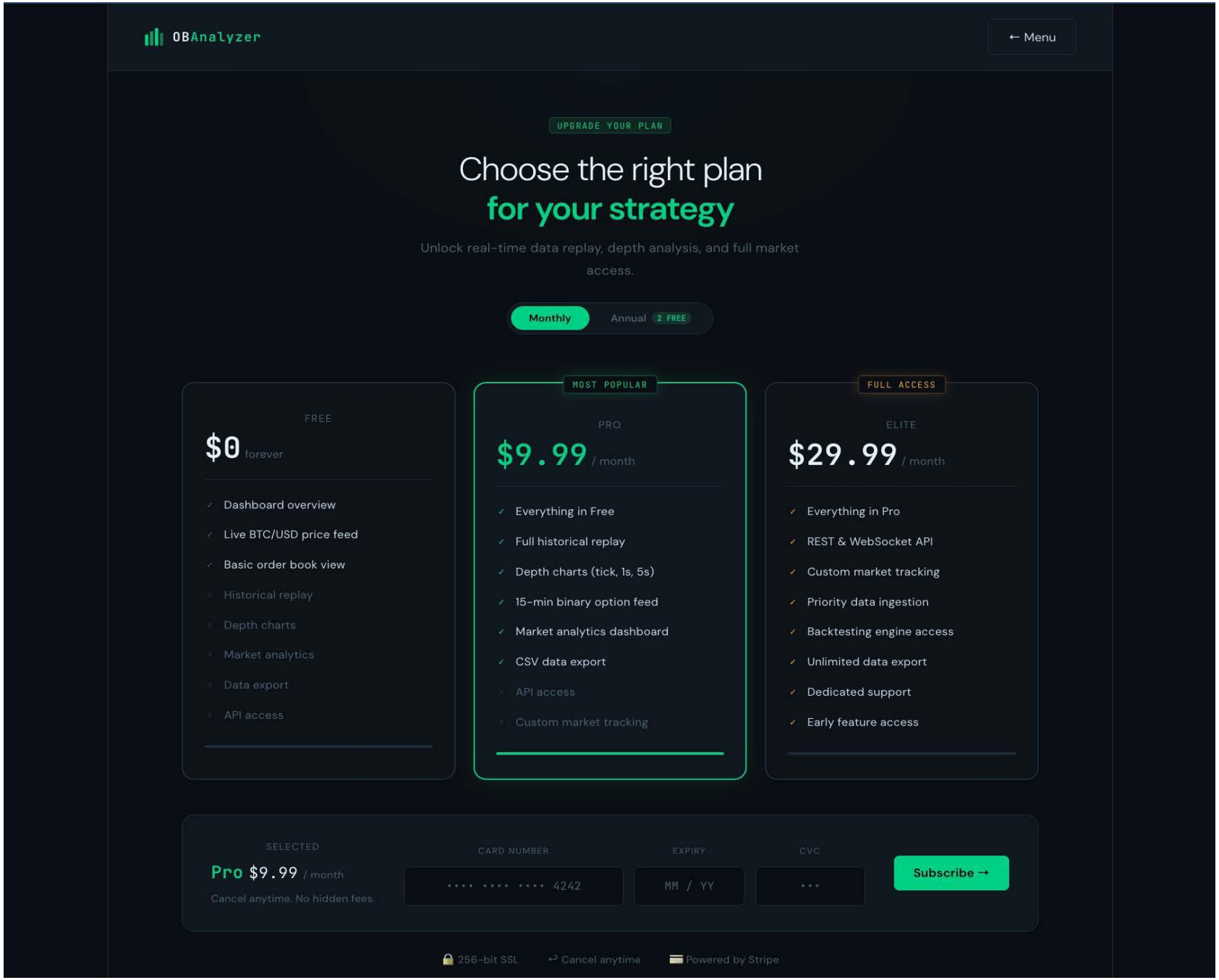
Prototype Screens

5- User information



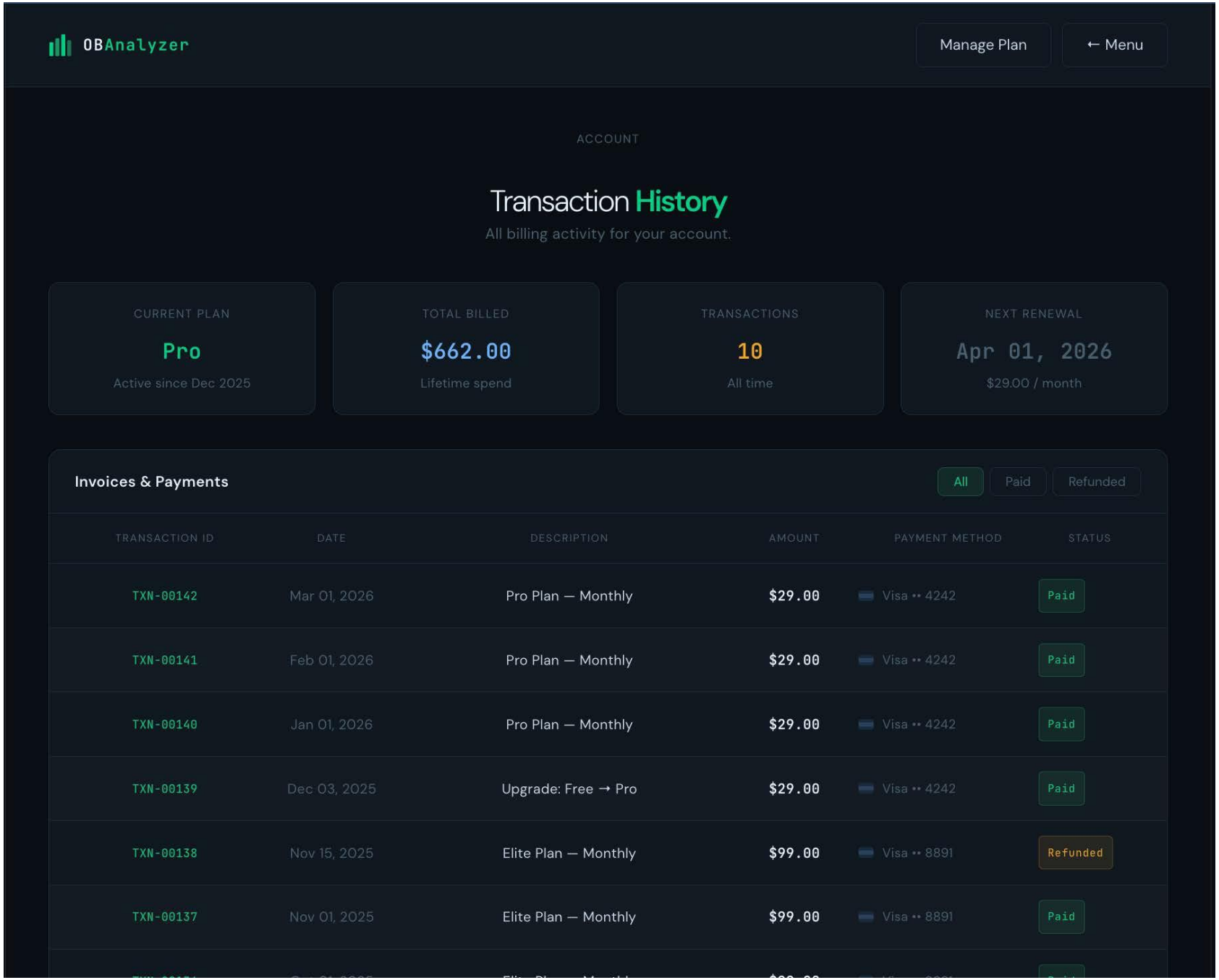
Prototype Screens

6- Buy product / service



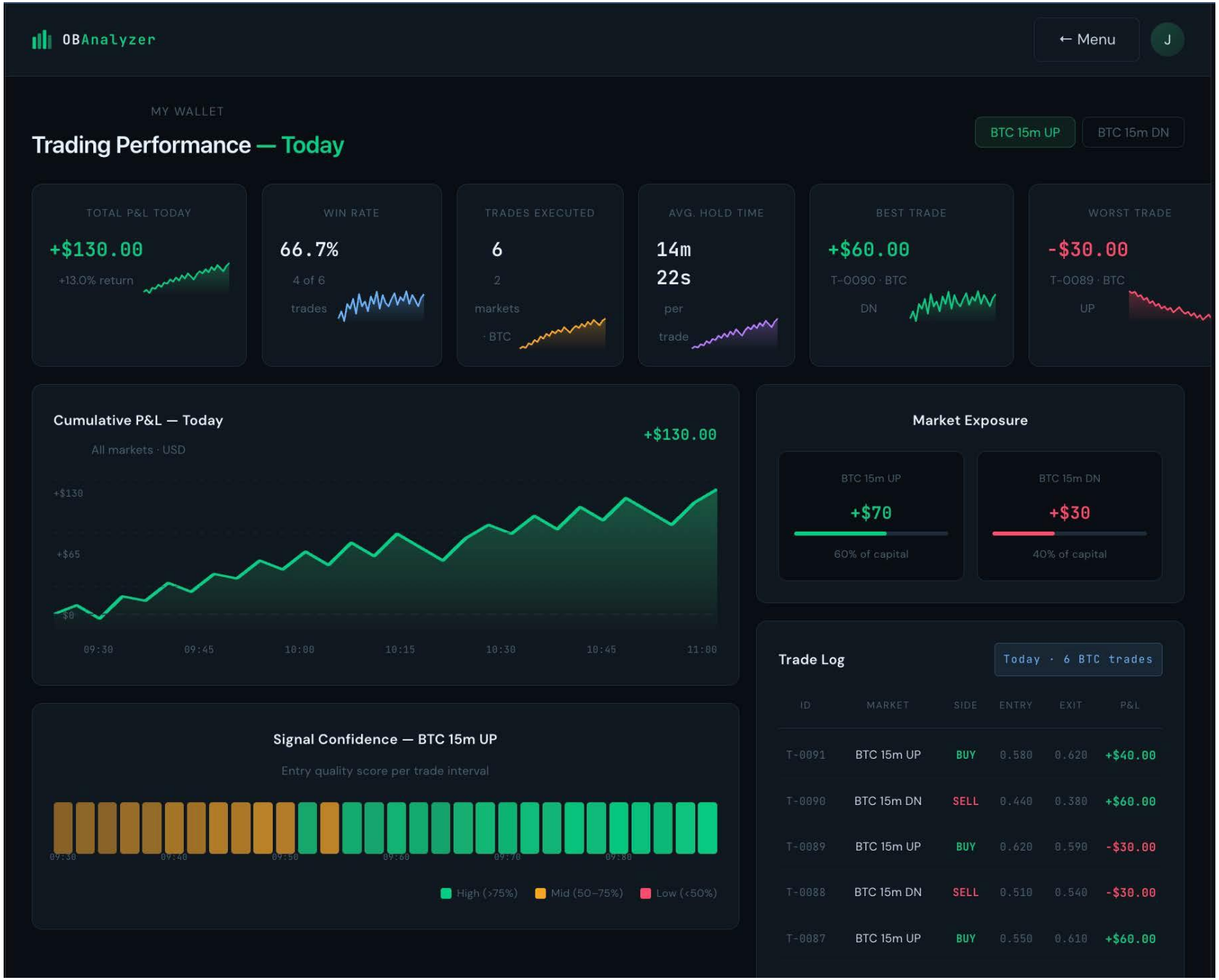
Prototype Screens

7- History screen to show past transactions



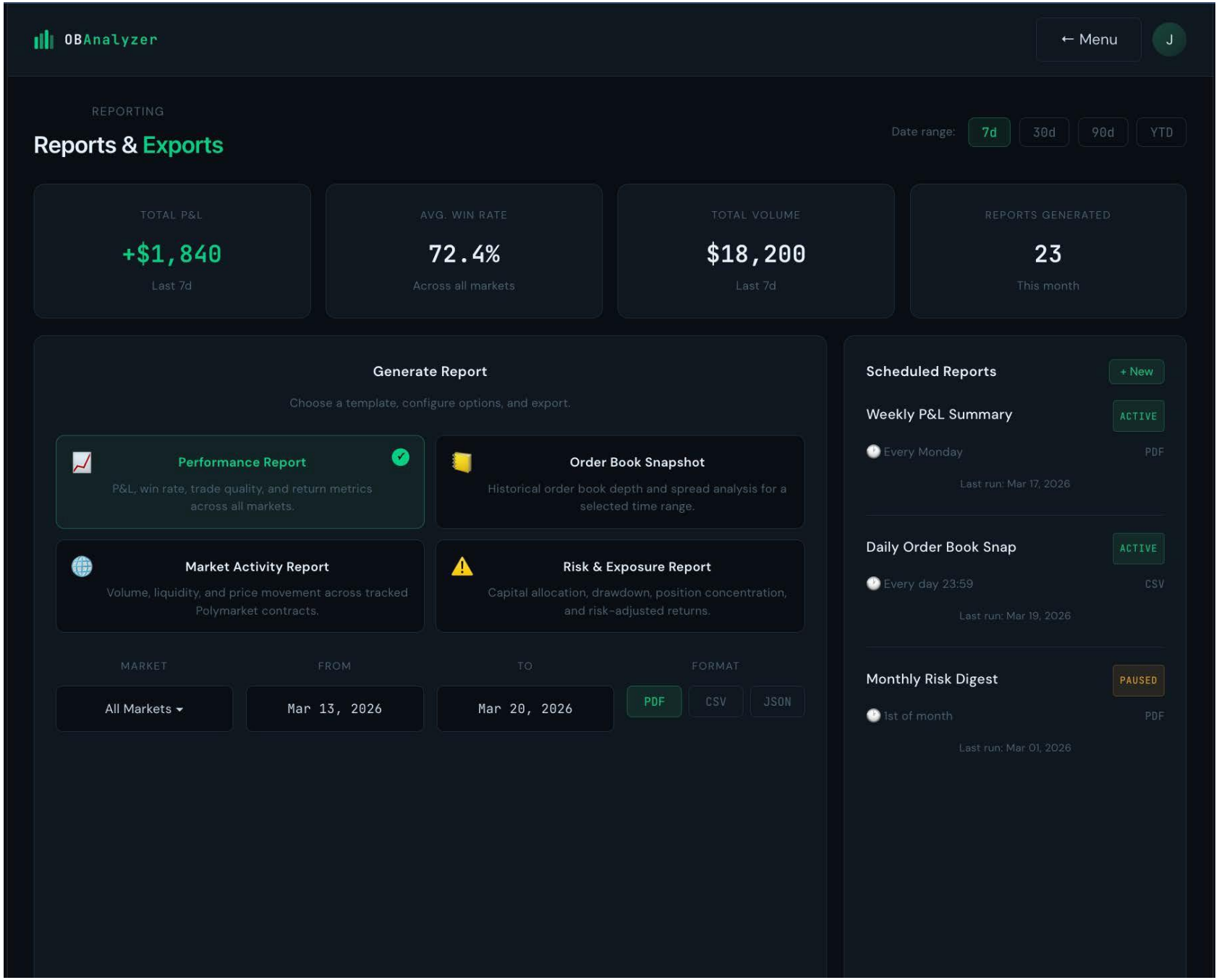
Prototype Screens

8- Analysis page of the user's current data



Prototype Screens

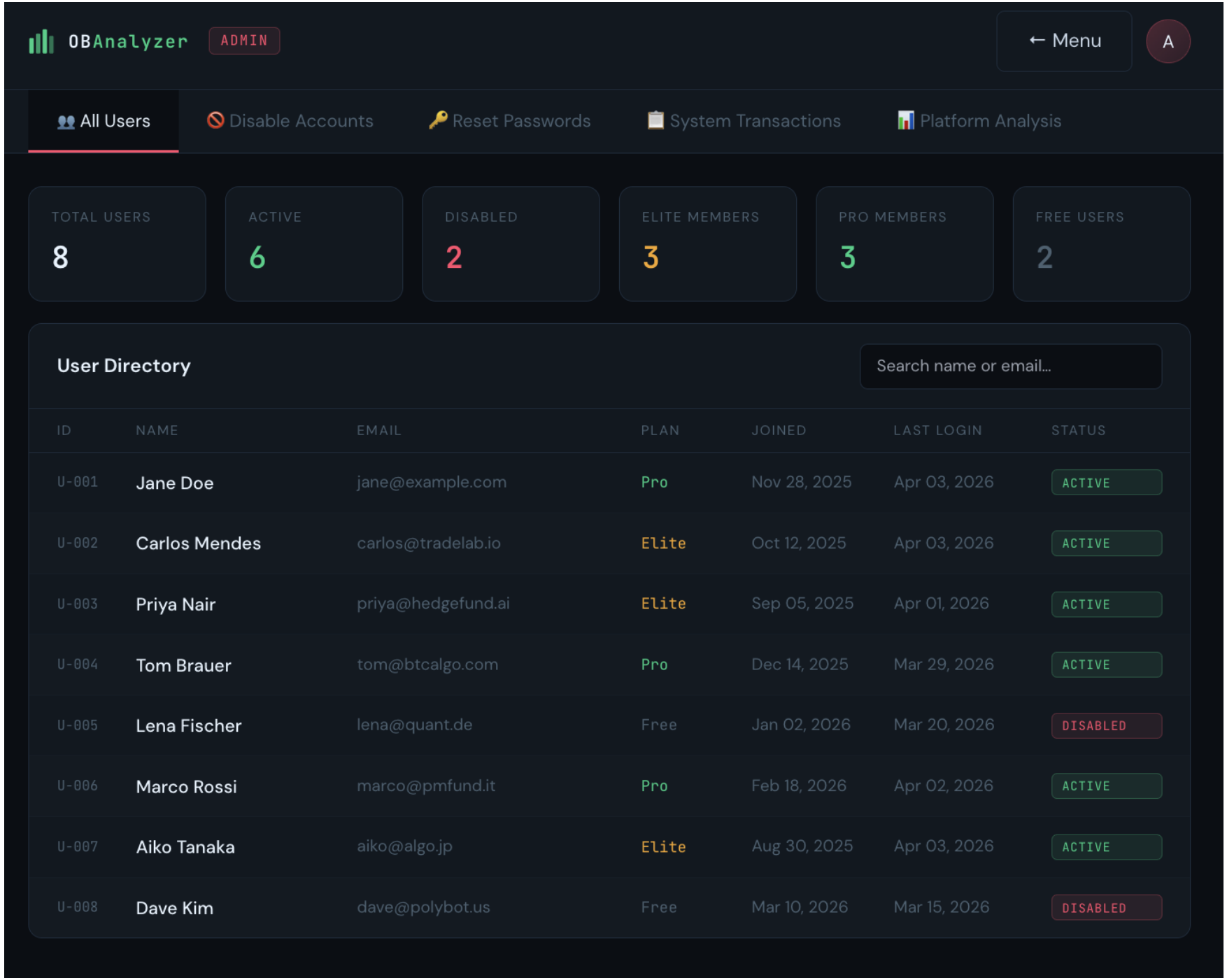
9 - Reporting functionality



Prototype Screens

10 – Admin

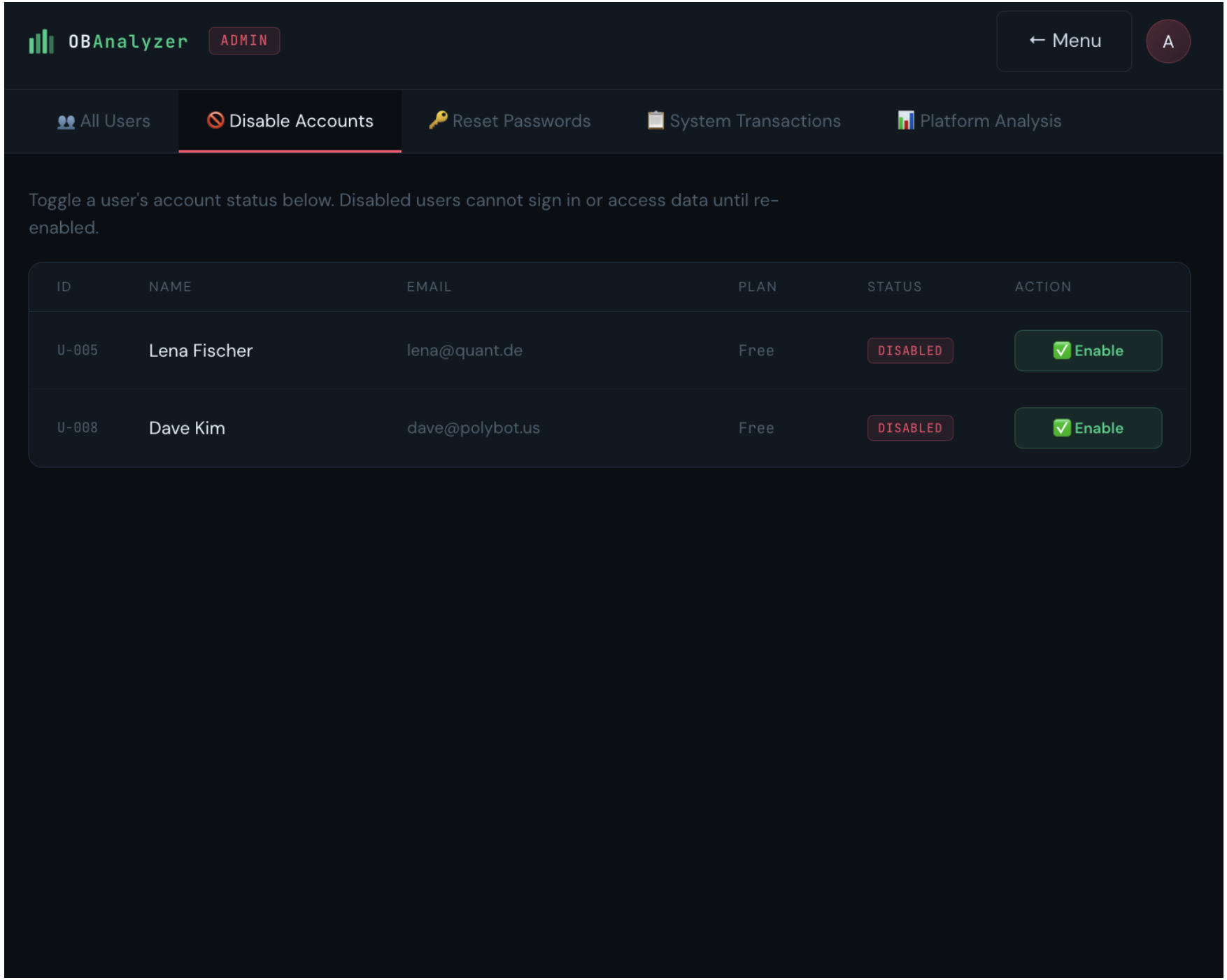
View All Users



Prototype Screens

11 – Admin

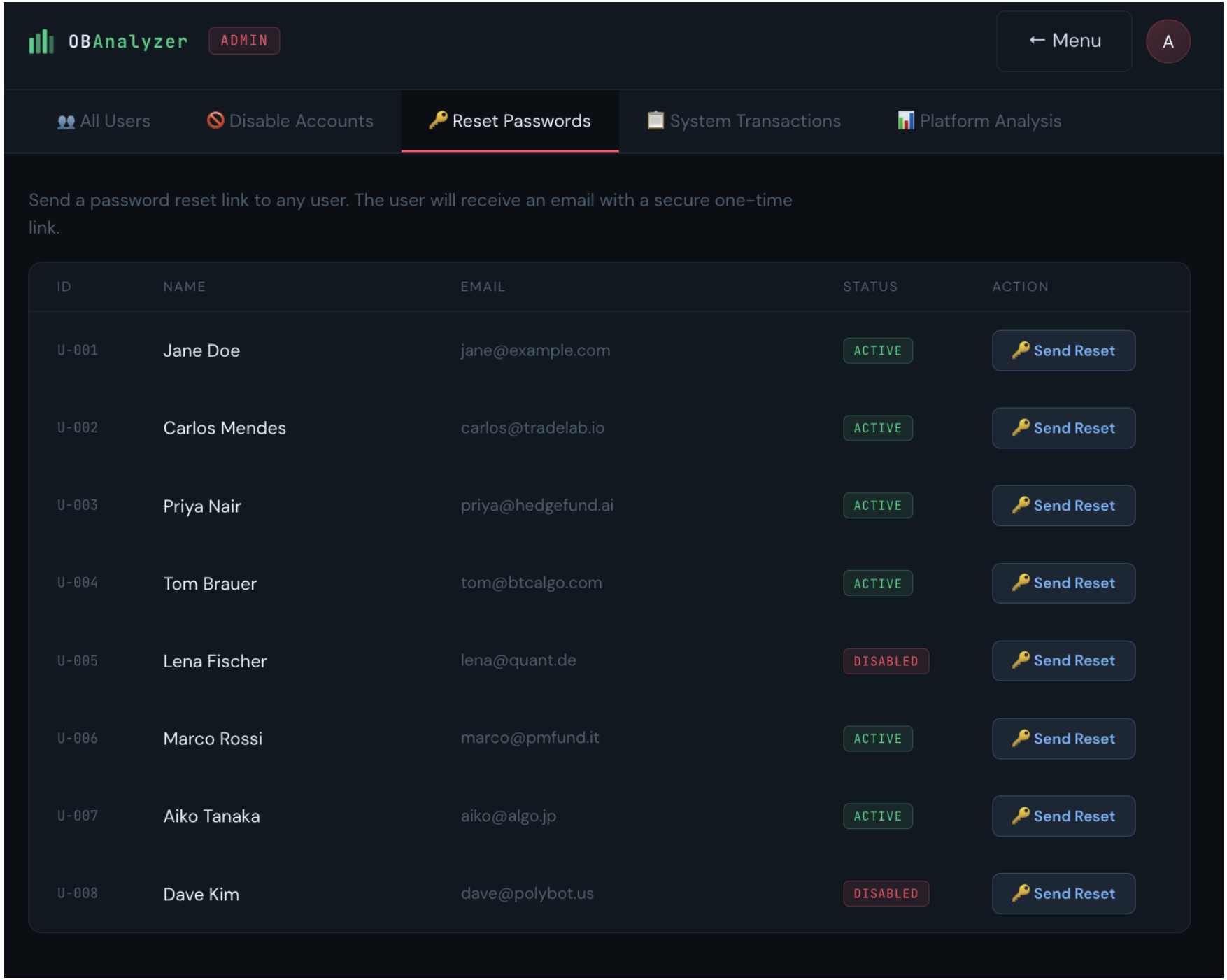
Disable user accounts



Prototype Screens

12 – Admin

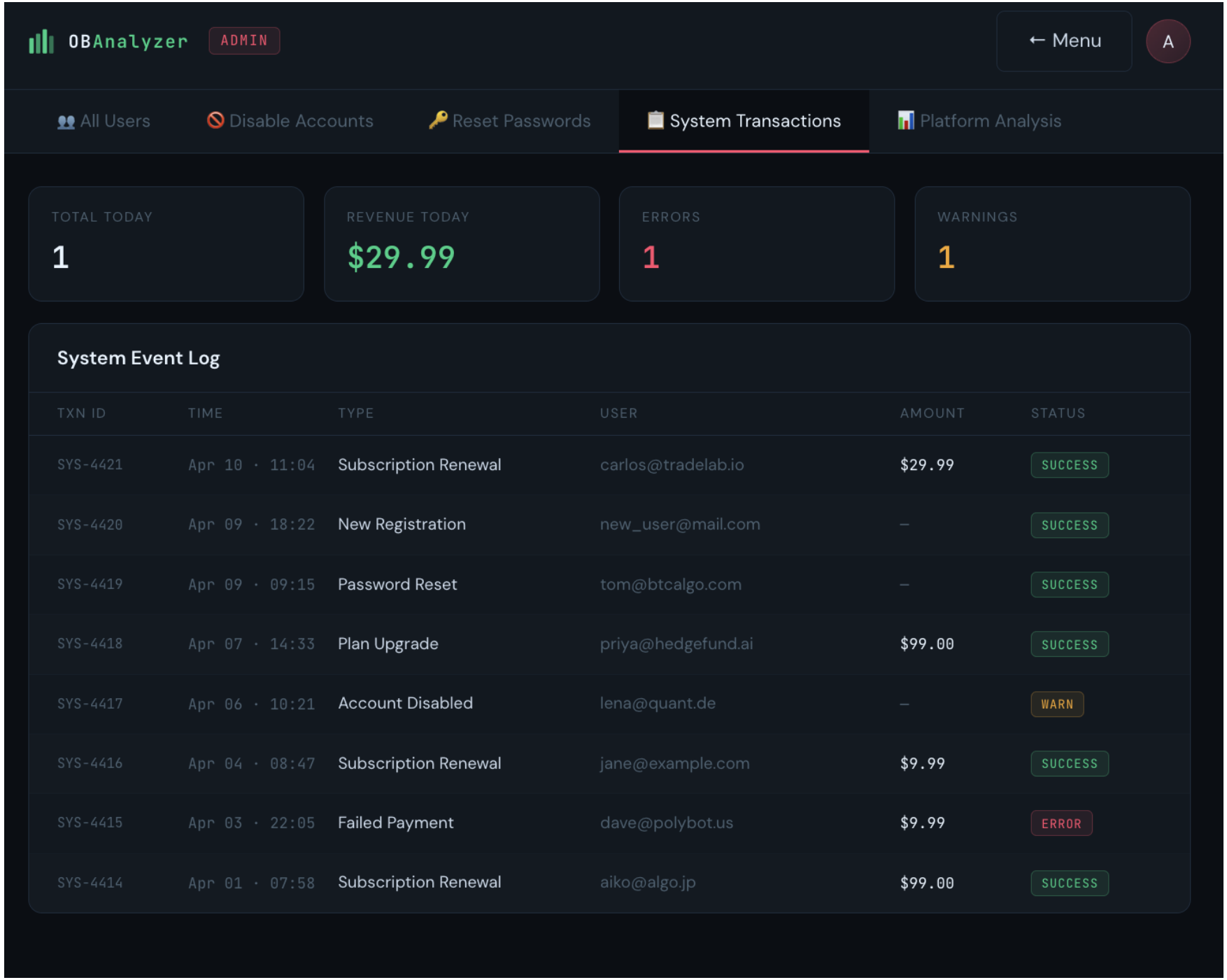
Reset user passwords



Prototype Screens

13 – Admin

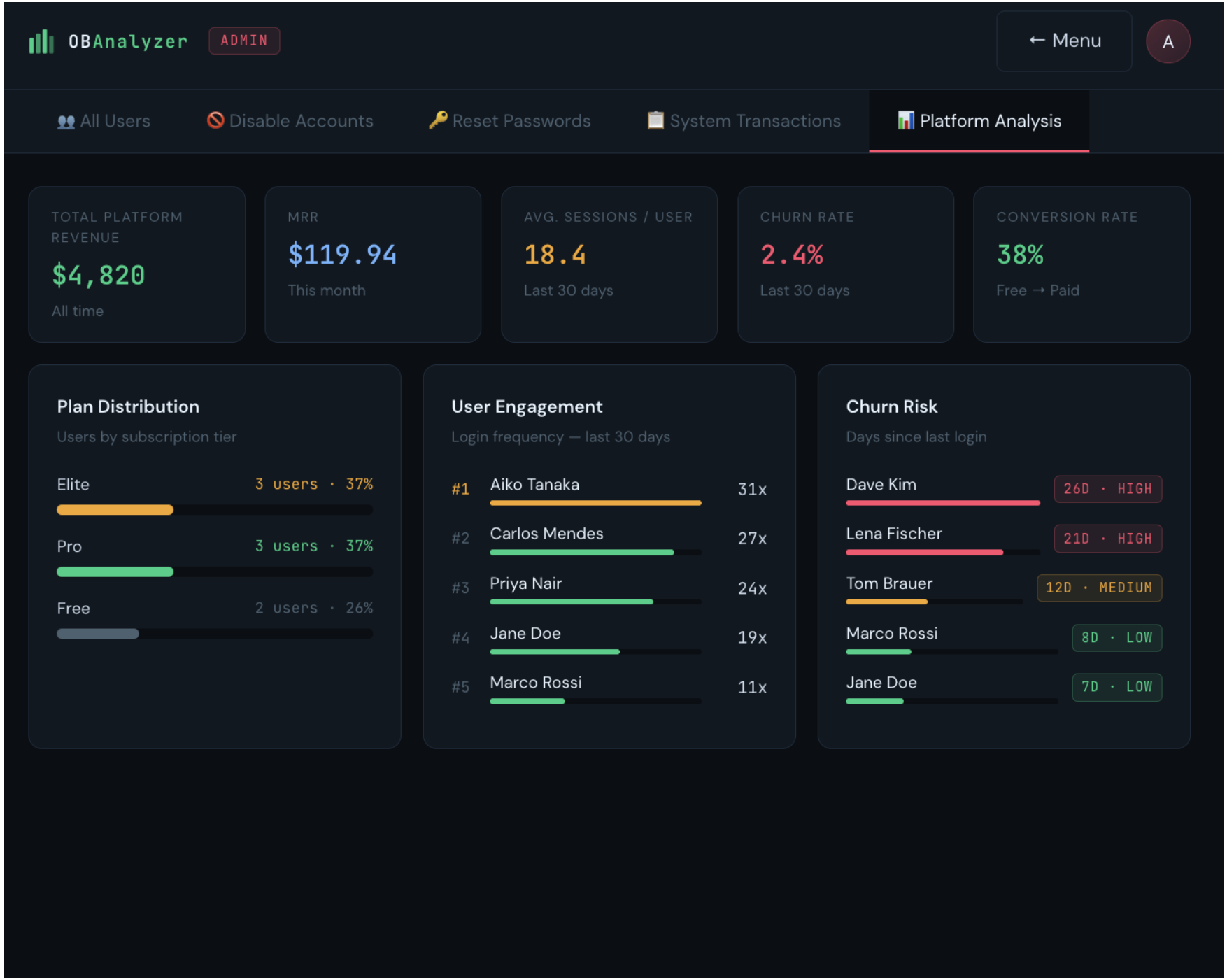
View “system” transact.



Prototype Screens

14 – Admin

Perform analysis across all accounts



Updated Design

Architecture

Data source: Polymarket.

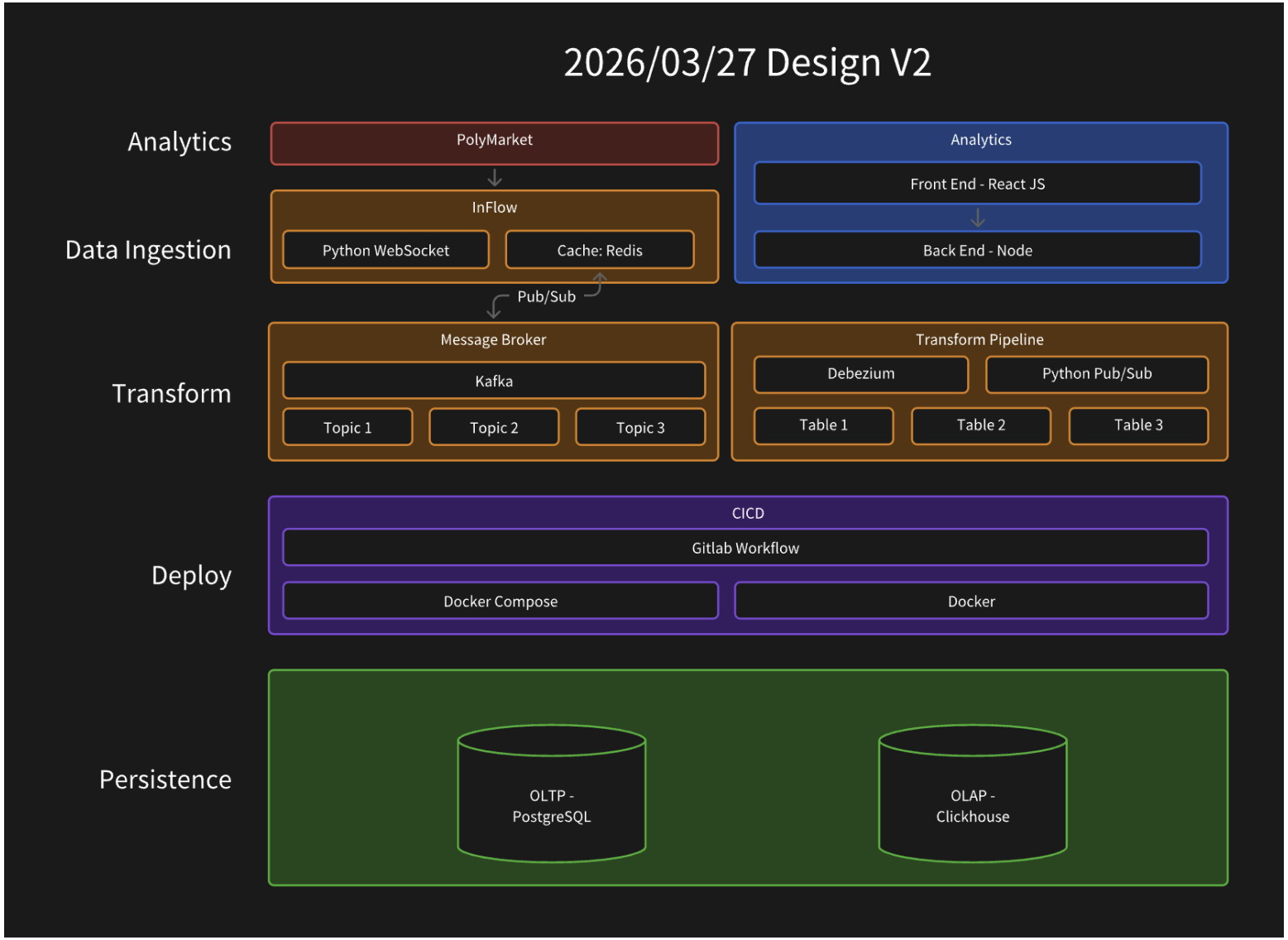
Ingestion layer: Python WebSocket collector with Redis cache.

Streaming backbone: Kafka message broker with multiple topics.

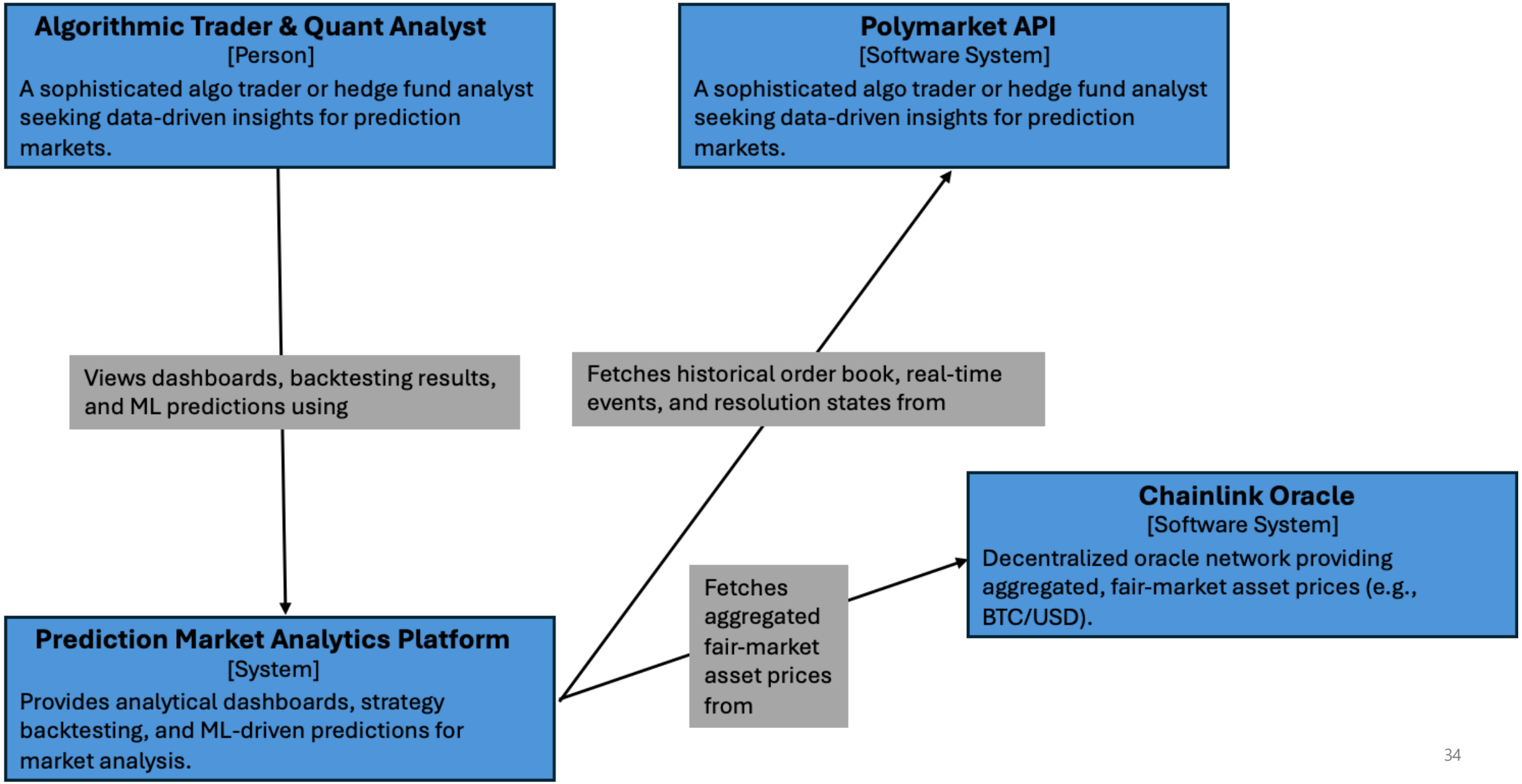
Storage: PostgreSQL (OLTP) for persisted normalized records.

Analytics backend: NodeJS services querying and aggregating stored data.

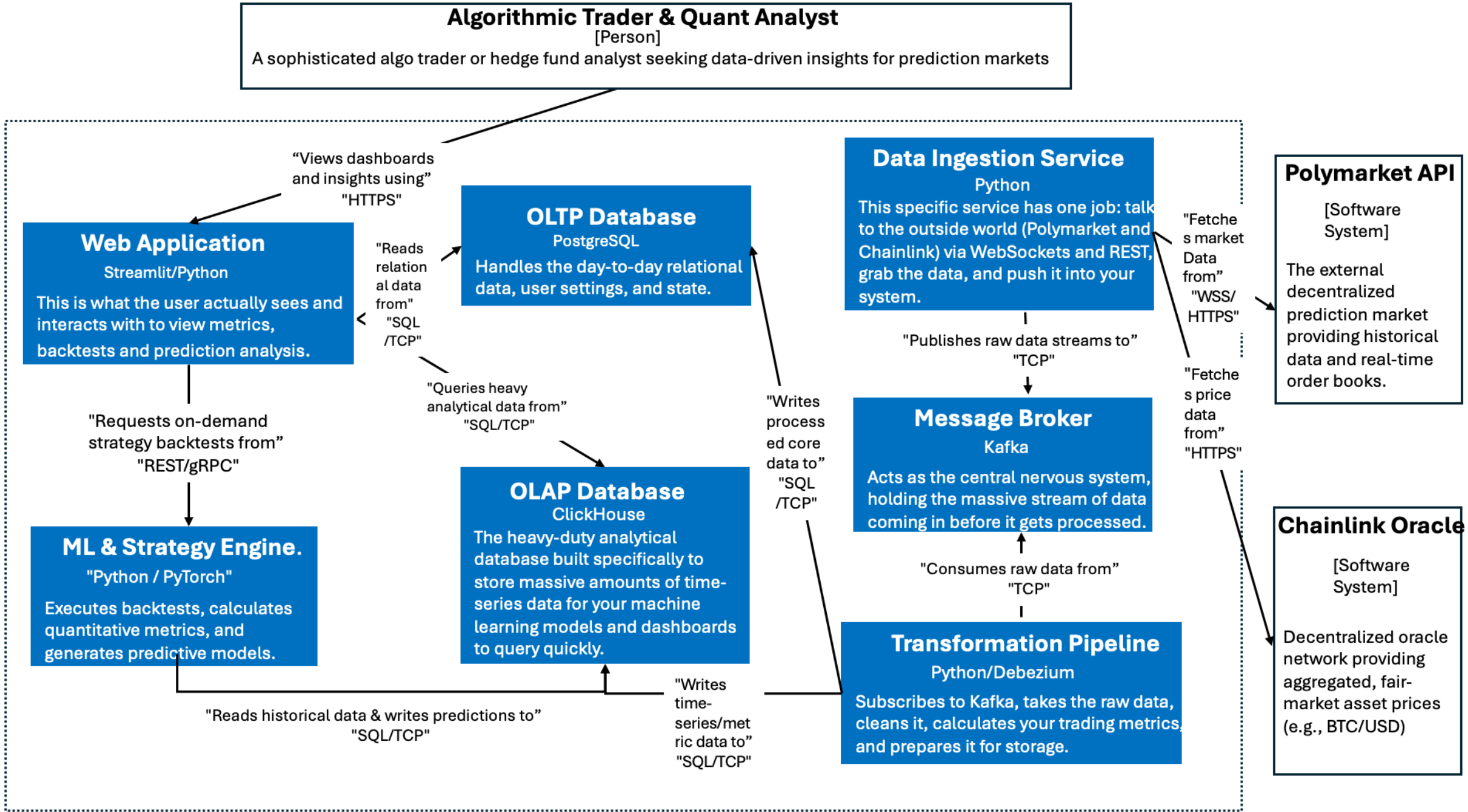
Analytics frontend: React UI for dashboards and research access.



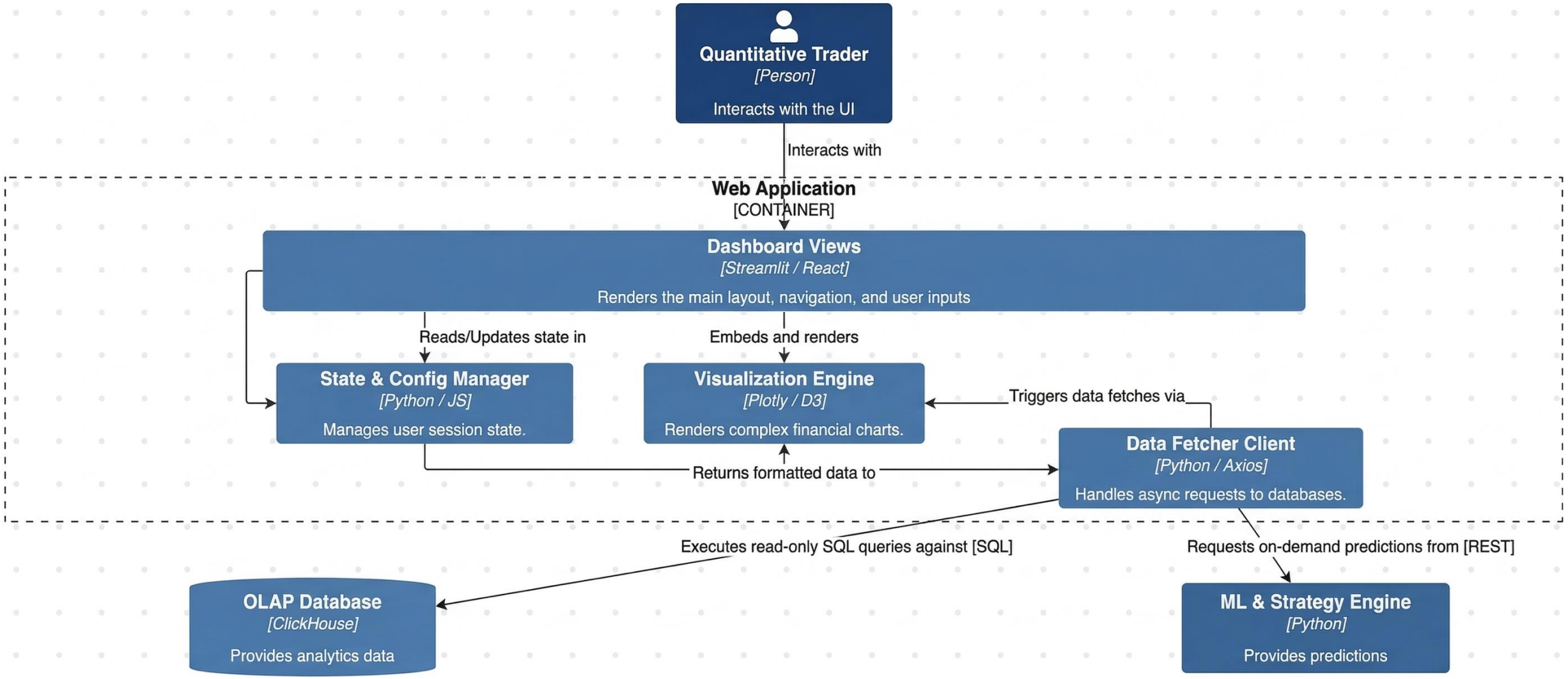
System Architecture (C4 model) - Level 1 Context



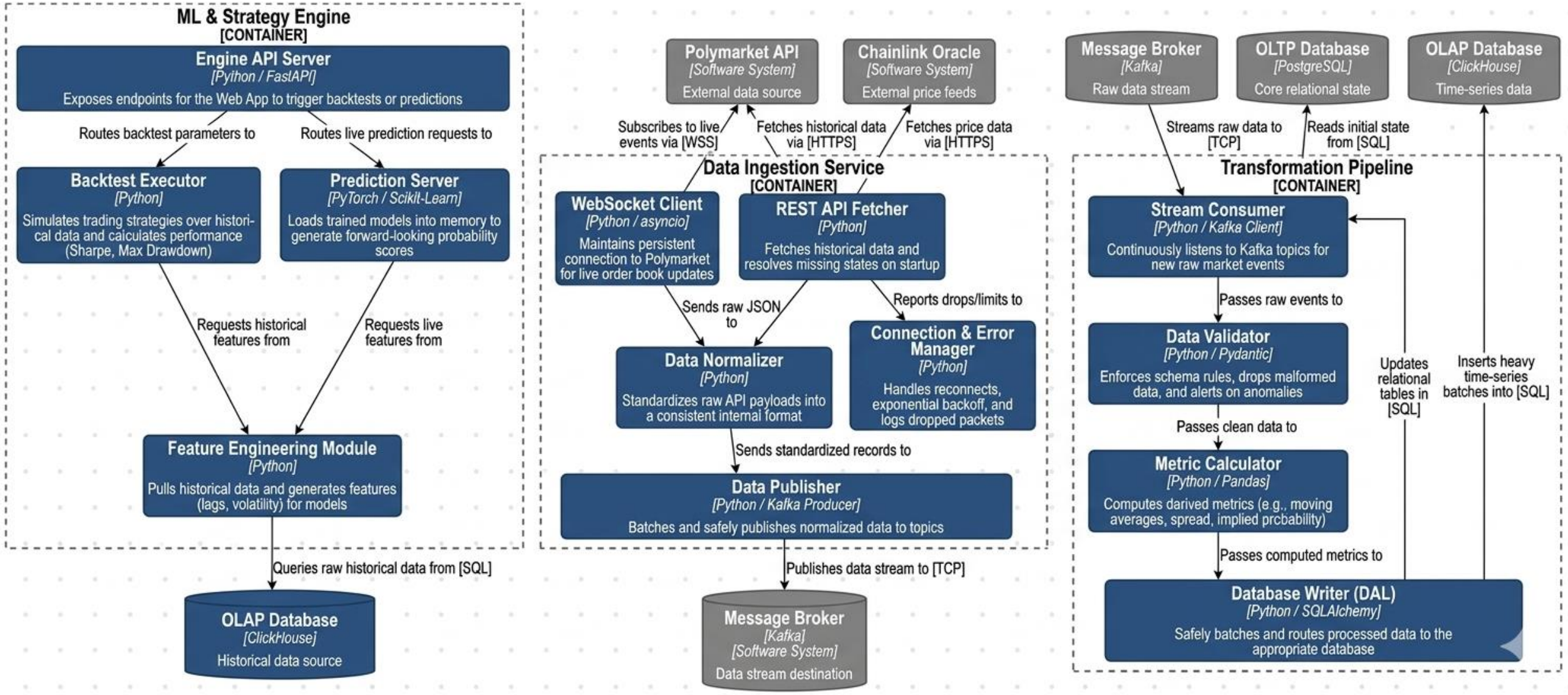
System Architecture (C4 model) - Level 2 Container



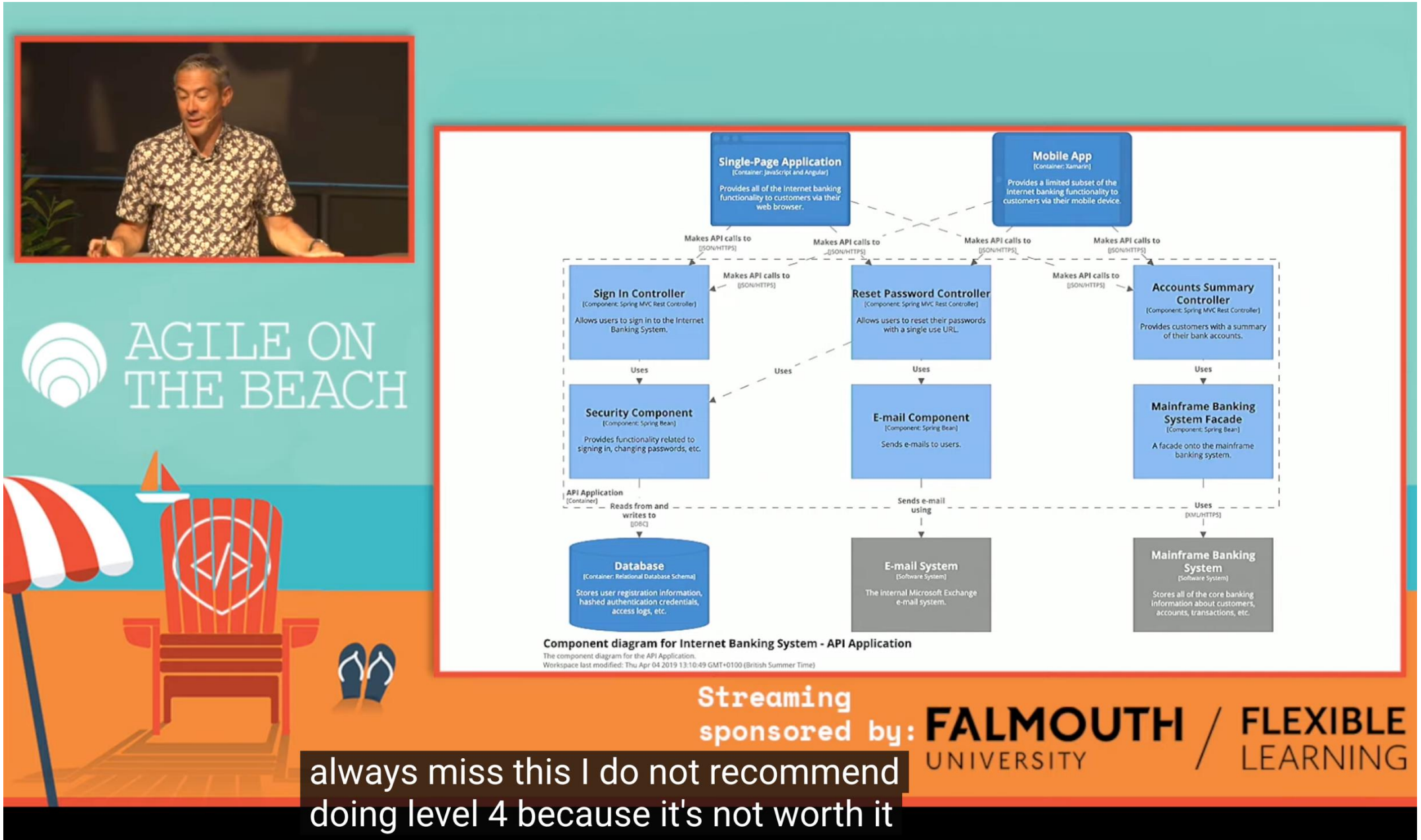
Level 3 Component – Web Application



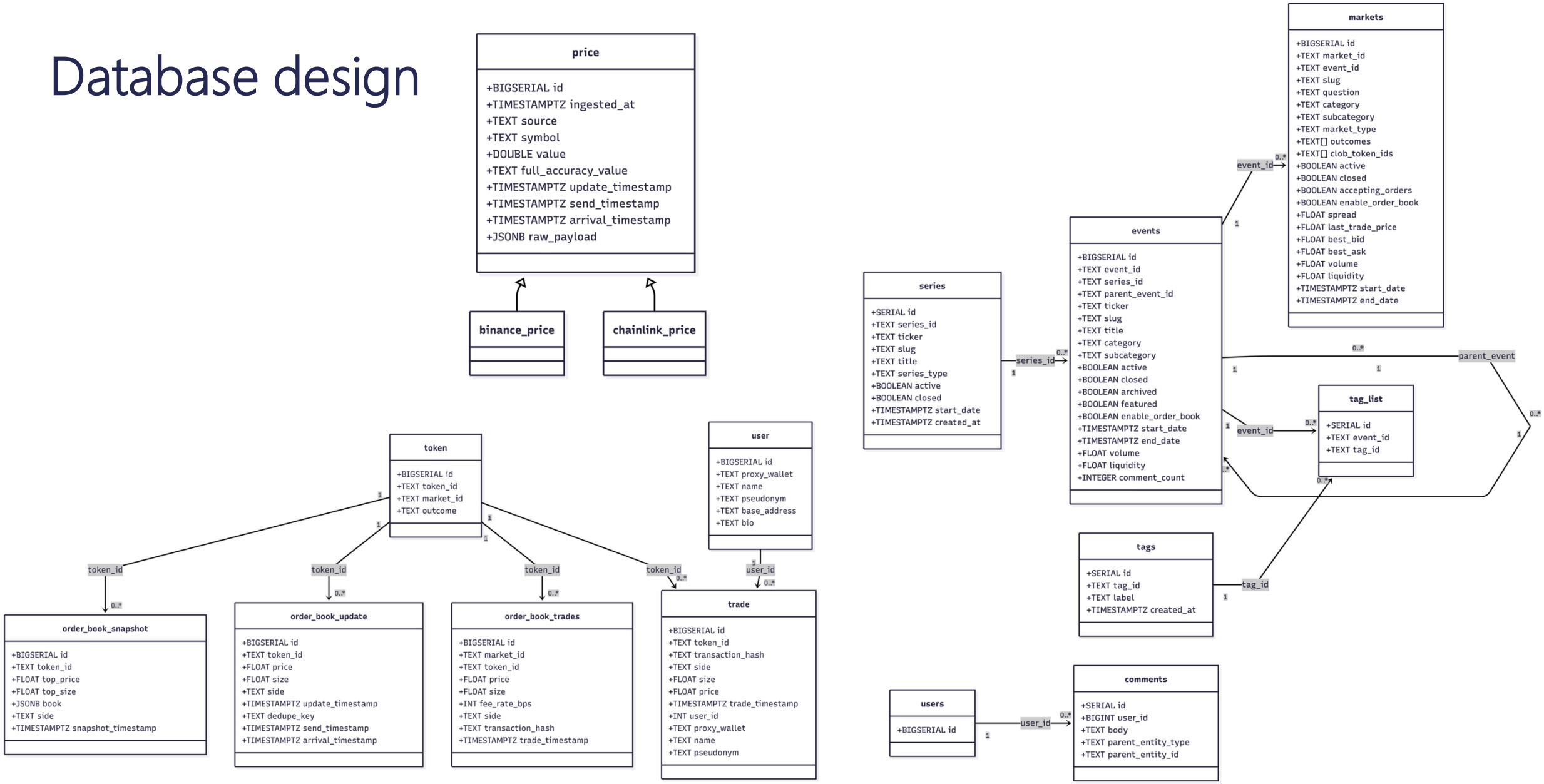
Level 3 Component – The Data Ingestion & Processing Containers



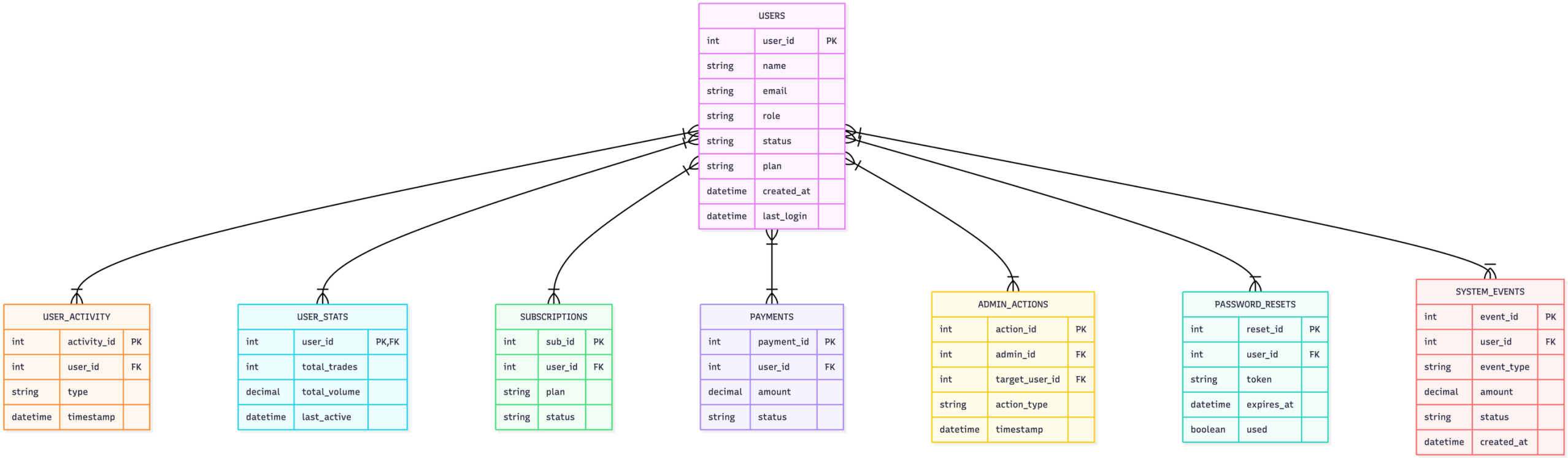
Level 4 Code


AGILE ON THE BEACH
Single-Page Application
[Container: JavaScript and Angular]
Provides all of the Internet banking functionality to customers via their web browser.
Mobile App
[Container: Xamarin]
Provides a limited subset of the Internet banking functionality to customers via their mobile device.
Makes API calls to
[JSON/HTTPS]
Sign In Controller
[Component: Spring MVC Rest Controller]
Allows users to sign in to the Internet Banking System.
Reset Password Controller
[Component: Spring MVC Rest Controller]
Allows users to reset their passwords with a single use URL.
Accounts Summary Controller
[Component: Spring MVC Rest Controller]
Provides customers with a summary of their bank accounts.
Uses
Security Component
[Component: Spring Bean]
Provides functionality related to signing in, changing passwords, etc.
E-mail Component
[Component: Spring Bean]
Sends e-mails to users.
Mainframe Banking System Facade
[Component: Spring Bean]
A facade onto the mainframe banking system.
API Application
[Container]
Reads from and writes to
[JDBC]
Sends e-mail using
[XML/HTTPS]
Database
[Container: Relational Database Schema]
Stores user registration information, hashed authentication credentials, access logs, etc.
E-mail System
[Software System]
The internal Microsoft Exchange e-mail system.
Mainframe Banking System
[Software System]
Stores all of the core banking information about customers, accounts, transactions, etc.
Component diagram for Internet Banking System - API Application
The component diagram for the API Application.
Workspace last modified: Thu Apr 04 2019 13:10:49 GMT+0100 (British Summer Time)
Streaming sponsored by: FALMOUTH UNIVERSITY / FLEXIBLE LEARNING
always miss this I do not recommend
doing level 4 because it's not worth it

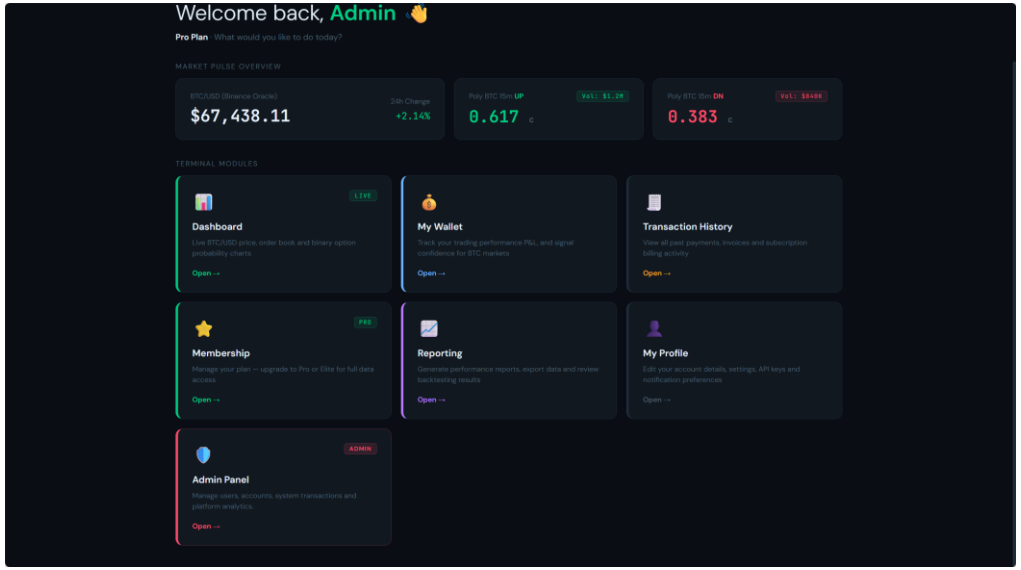
Database design



Database schema for users



Logging and Monitoring



OWASP Guideline	Decision	Rationale
Application code is the primary event data source	Implemented	All log events are emitted from application code (frontend JS + backend Express middleware), not inferred from infrastructure logs alone
Use a separate partition/database account for log storage	Implemented (backend)	Backend writes logs to a dedicated `ob_audit_logs` table using a restricted DB user with INSERT-only permissions on that table
Do not expose logs in web-accessible locations	Implemented	Log endpoints are admin-only (`/api/analytics/admin/*`), protected by `authenticate` + role check middleware. No log files are served as static assets
Use standard formats over secure protocols when sending logs to other systems	Implemented	All log entries use structured JSON (see Section 3). In-transit log shipping uses HTTPS only
File-based logs: apply strict directory/file permissions	Not applicable	This project uses a database-backed log store, not flat files. If file logging is added in future, permissions will be enforced
Store/copy log data to read-only media as soon as possible	Deferred	Outside scope of current sprint. Planned: replicate `ob_audit_logs` to an append-only S3 bucket in a future sprint
All access to logs must be recorded and monitored	Implemented	Admin reads of the log table are themselves logged as `admin_log_access` events
Restrict log read privileges; review periodically	Implemented	Only users with `role = 'admin'` can query log data. Privilege review cadence: monthly

Where to Record Event Data:

- Primary source: Application code (Frontend JS + Backend Express).
- Storage: Append-only ob_audit_logs database table.
- Access: Admin-only endpoints, restricted DB user (INSERT only).
- Transmission: Structured JSON over HTTPS.

Which Events to LOG:

- All login attempts (success + failure)
- MFA verification outcomes.
- Admin panel access + redirects
- Order execution (attempt / success)
- Report generation + subscription changes.
- Unhandled client JS errors (window.onerror)

LOG entry Format:

- Datetime – ISO 8601 UTC (server-stamped)
- Appid – level – event (snake_case)
- Userid (opaque ID, not email/PII)
- userRole – sessionid – sourceIp
- Endpoint – outcome – description
- Requestid for cross-service tracing.

Optionally logged: sensitive data reads (wallet, transactions) at DEBUG level for anomaly detection.

Not logged: static assets, WebSocket tick updates, UI toggle state — no security value, would add noise.

Event & Attribute Design

Always log

Event category	Specific event	Implemented in
Input validation failure	Malformed JWT / missing auth header	`authenticate.js` middleware
Authentication success	User login via `/v1/auth/verify-mfa`	`authController.js`
Authentication failure	Wrong password, wrong MFA code	`authController.js`
Authorisation failure	Non-admin user attempts admin endpoint	`requireSubscription.js` + role check
Session management	Token issued, token revoked on sign-out	`authController.js`
Subscription / billing events	Plan upgrade, payment processed, payment failed	`billingController.js` (planned)
Data export	CSV/PDF report generated and downloaded	`analyticsController.js`
Admin actions	Account disabled, password reset triggered	`analyticsController.js` admin routes
Application errors	Unhandled exceptions, 5xx responses	Global error handler in `app.js`

Optionally log (implemented)

Event category	Decision
Successful read of sensitive data (wallet overview, transaction history)	Logged at DEBUG level — useful for anomaly detection
Order execution	Logged — financial action with non-repudiation value
Market/contract switch on terminal	Not logged — no security or business value; would add noise

Explicitly not logged

Event	Reason not logged
Static asset requests (HTML, CSS, JS files)	Infrastructure-level concern; no security value at application level
Chart data polling / WebSocket tick updates	High-frequency, no auth risk, would fill logs with noise
UI toggle state (Area on/off, chart tab switch)	Pure UI state, no security or business relevance
Passwords, raw MFA codes, card numbers	OWASP explicitly prohibits logging credentials and payment data

OWASP attribute	Field in our schema	Notes
When (timestamp)	`datetime`	ISO 8601 UTC. Server time only — not client time
Where (application identifier)	`appid`	Fixed value `obanalyzer.api`
Where (geolocation / source IP)	`sourceIp`	Extracted from `req.ip` after proxy trust configuration
Who (user identity)	`userId`	Internal user ID (not email — avoids PII in logs)
Who (user role/permissions)	`userRole`	`user` or `admin`
Who (session)	`sessionId`	Hashed session identifier
What (event type)	`event`	Snake_case verb — follows OWASP Logging Vocabulary where applicable
What (endpoint)	`endpoint`	HTTP method + path
What (outcome)	`outcome`	`success`, `failure`, `error`
Severity / level	`level`	`DEBUG`, `INFO`, `WARN`, `ERROR`, `CRITICAL`
Human description	`description`	Free text, sanitised before write
Trace correlation	`requestId`	UUID generated per request in middleware

Log Entry Format — Event Attributes (OWASP Schema)

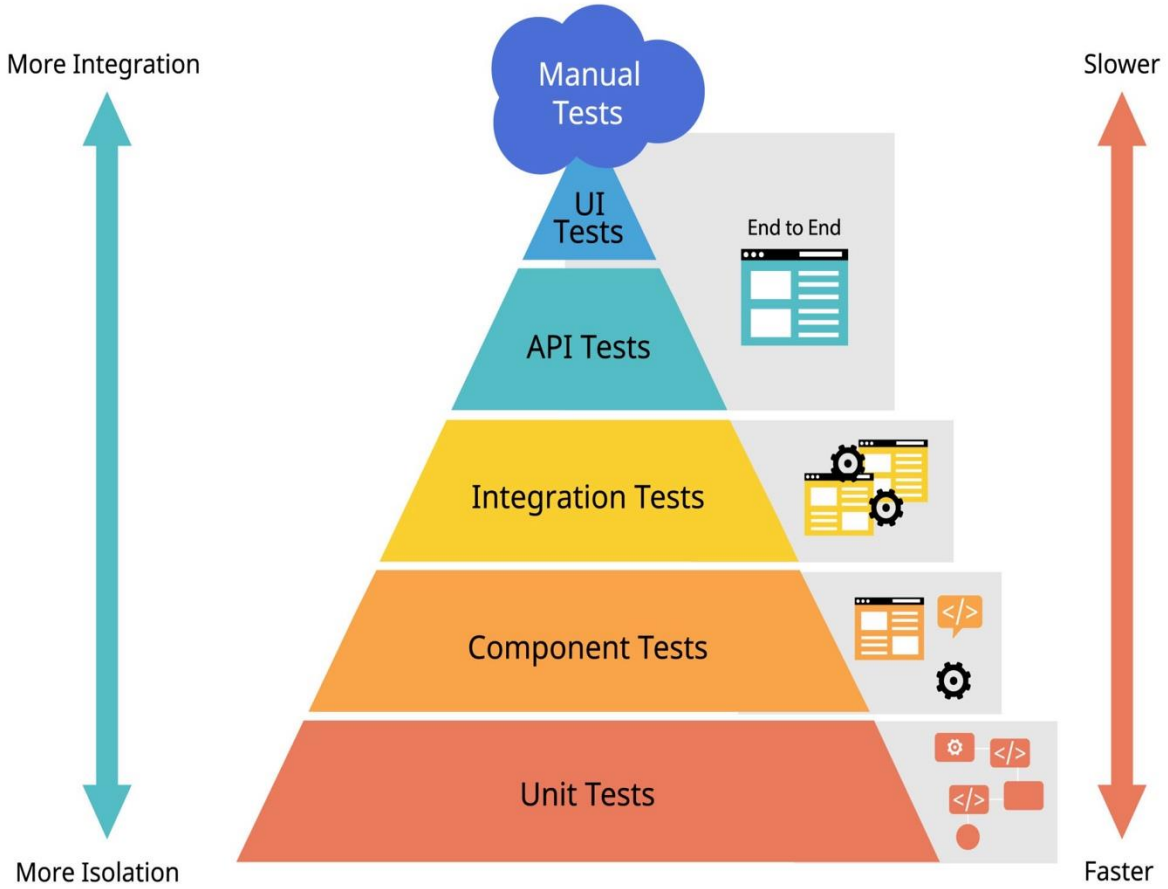
All entries are JSON. Server stamps datetime — client time is advisory only.
Passwords, raw tokens, email addresses, and card numbers are never written to logs.

All string fields sanitised before write (CR / LF / delimiter characters stripped).
Logging failures are silently swallowed — never crash or block the application.

Reference: OWASP Logging Cheat Sheet · C9: Implement Security Logging and Monitoring

Testing approach Methodology

Unit Tests (The Foundation):
Testing individual functions in isolation.
Task: Does this specific Python function correctly parse a Polymarket probability format into a float?
Integration Tests (The Middle):
Testing whether decoupled system modules communicate correctly.
Task: Does the WebSocket ingestion service correctly write data to the cache? Does the message broker correctly ingest and forward the event?
Data Quality & Contract Tests (Pipeline Specific):
Testing whether schema or upstream data changes are handled safely.
Task: If Polymarket adds a new field to their JSON response, does our pipeline safely ignore it or crash?
End-to-End (E2E) Tests (The Peak):
Testing whether the full system works correctly from input to final user-visible output.
Task: If a trade event happens on the mock Polymarket WebSocket, does it successfully propagate through the pipeline, get stored in OLAP database, and appear correctly on the dashboard?



Testing approach In Data Ingestion

Corner Cases

Network Jitter & Disconnects.
What happens if the Polymarket WebSocket drops packets or silently disconnects? Does the system recognize the stale state?

Malformed Payloads.
Polymarket sends an unexpected null value for a betting pool size, or a string instead of an integer.

Redis Memory Eviction.
If the downstream pipeline slows down and Redis fills up, do we drop new data or block the WebSocket?

Examples from unit test to E2E test

Data Parsing
Objective: Verify that raw JSON from Polymarket is correctly transformed into our internal Python Data Class / Pydantic model.
Implementation: We do not need an internet connection for this. We save a sample Polymarket JSON response as a local fixture file. We write a pytest that feeds this static JSON into our parser function.

Python InFlow to Redis
Objective: Verify that parsed data is successfully cached in Redis with the correct Time-To-Live (TTL).
Implementation: We use Testcontainers (a framework that spins up a lightweight, throwaway Docker container of Redis just for the test). Our Python code connects to this temporary Redis instance, writes a mock message, and reads it back.

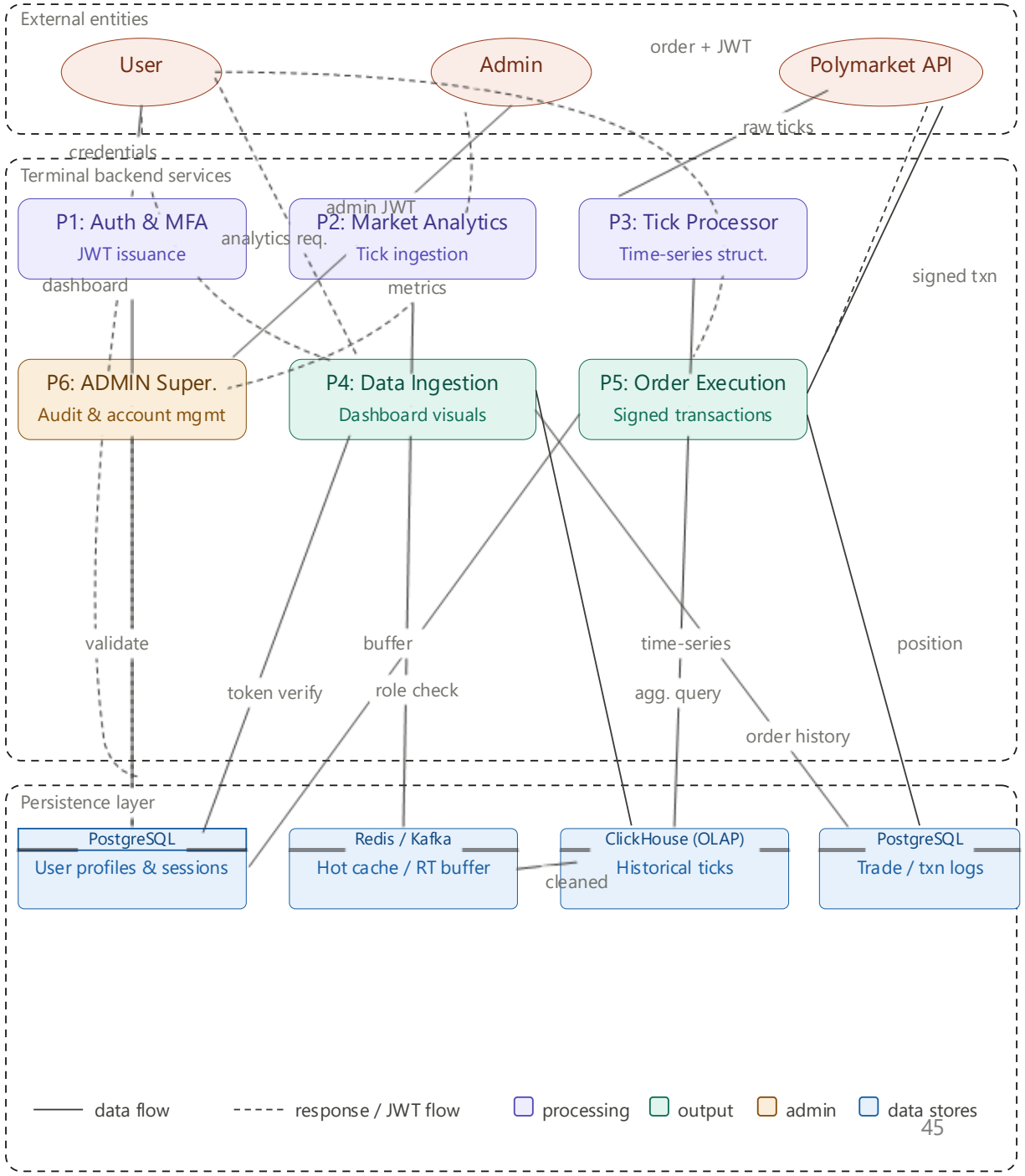
The Ingestion Slice
Objective: Verify the entire Inflow module works from WebSocket to Message Broker.
Implementation: We create a mock WebSocket server that mimics Polymarket and emits pre-determined trade events at a specific rate. We spin up our Python InFlow service, a Redis instance, and a single Kafka node using Docker Compose.

Data Flow Diagram (DFD)

System Overview:

- Users interact with the system through frontend connected to backend APIs
- Backend services process authentication, logics and provide database access
- External market data – Polymarket is ingested, processed and stored
- Data views as dashboards and analytics components.

- **Assets:** User credentials and Oauth tokens, user permissions, Orderbook data and trade records, market structure data, internal APIs and service endpoints, PostgreSQL database contents, external market data (Polymarket based), system logs, audit records.
- **Actors:** User, Administrator, Polymarket API, Backend Services (for handling logics and data processing)
- **Threats: Based on STRIDE analysis**



Data Flow Diagram (DFD) – STRIDE analysis

All | S | T | R | I | D | E

CATEGORY	SEV.	NODES	THREAT	MITIGATION
Spoofing	H	User P1	JWT/session token stolen & replayed	Short-lived tokens + revocation list in D1
Spoofing	H	PolyAPI P2	Polymarket API spoofed, fake tick data injected	TLS cert pinning + HMAC signature on every feed
Tampering	H	P5 PolyAPI	Order request tampered before signing	User-signed payload; P5 verifies hash against receipt
Tampering	H	P2 D2	Tick data corrupted in Redis/Kafka buffer	Per-message checksum; P3 validates before writing D3
Repudiation	H	P5 D4	User denies placing order — no audit trail	Append-only signed order log in D4
Repudiation	M	P6 D1	Admin denies executing command	Immutable admin audit log; ship to off-system SIEM
Info disclosure	H	D1 P1	User PII leaked via errors or SQL injection	Parameterised queries; encrypt PII columns; no stack traces
Info disclosure	H	P6 PolyAPI	API keys exposed via overexposed admin endpoint	P6 behind VPN/allowlist; secrets manager for all keys
Denial of service	H	P2 D2 P3	Tick flood overwhelms data pipeline	Rate-limit at P2; back-pressure on Kafka; circuit breaker
Denial of service	M	P4 D3	Expensive analytics queries degrade OLAP store	Per-user query budget + result caching in P4
Elevation of priv.	H	P6 P4 P5	Regular user JWT accepted on admin P6 endpoints	Deny-by-default middleware; role validated on every request
Elevation of priv.	H	P5 D1	JWT role claim tampered to bypass order checks	RS256 signing; re-fetch authoritative role from D1

STRIDE analysis:

- Spoofing
  - Stolen session or OAuth token, external data source is not true.
- Tampering
  - Modified API requests, data corruption and wrong market data.
- Repudiation
  - Users/admin deny actions, missing logs for audit.
- Information disclosure:
  - Data leakage like user information and API keys, and overexposed admin endpoints.
- Denial of service
  - Data pipeline overload.
- Elevation of privilege:
  - Users access admins endpoints, and lack of authorization check.

Data Flow Diagram (DFD) - Team Discussion

'What are we working on?'
Can we see where all the data goes and who uses it?
- Yes, we can see all the data goes and the actors by nodes connections, and comments between those node represent the relations.
- The entities interact with which endpoints are clear.

Do we see the processes that move data from one data store to another?
- Yes, the diagram shows exactly how backend processes data in transit.
- For example, the Tick processor (P3) taking buffered data from Redis/Kafka and moves TS structure to Clickhouse OLAP store.

Does the diagram show all the trust boundaries, such as where different accounts interact?
- Yes, the bounding box separates boundaries by "External Entities", "Terminal backend Services", and "Persistence Layer".

Can we look at the diagram and see exactly where the software will make a security decision?
- Yes, a security decision will be made by Auth & MFA (P1), and Admin Super (P6) as authentication decisions and authorized role checking processes run.

Does the diagram reflect the current or planned reality of the software?
- Yes, the software is well-defined by DFD.
- Including specific database including PostgreSQL, ClickHouse, Redit, and external API (Polymarket).

'What can go wrong?'
Have we looked at each element of the diagram?
- Yes, we have fully reviewed the API gateways, the internal backend processes from P1 to P6 and the data storages.

Have we looked for each of the STRIDE threats?
- Yes, we have fully reviewed each threat by STRIDE analysis, by that two vulnerabilities for every single category are being checked.

For each threat, can we describe the actor, the attack, and the impact?
- Yes, we table has defined the nodes involved, with severity (High/Median), and the nature of attack such as stealing a JWT token or attacker flooding within the pipeline.

'What are we going to do about it?'
Is there a proposed/planned/implemented mitigation to address each threat?
- Yes, we have mitigations for each threat based on STRIDE and also included in the table.

Are there test cases we should write?
- Testing our API endpoints for SQL injection vulnerabilities.
- Simulating tick data floods to ensure P2 rate, which limiting and Kafka back-pressure hold up.
- Attempting to access P6 administration endpoints with regular user JWTs to verify our deny-by-default middleware.

Testing approach - Unit test suite for Full Stack

Corner Cases

Invalid User Inputs. (frontend validation)
A user submits invalid format of forms, registration info, or empty data. The system needs to prevent submission and provide clear validation feedback in the UI level.

Unauthorized Access. (security)
A user tries to access admin routes or attempt to get access to API keys. Access control should timely focus on across both frontend and backend (render and authorization).

API Failure, Timeout or Error Response.
The backend may return an error or slow responses during user requests. The frontend should handle those scenarios without crashing or exposing internal errors.

Invalid Request, Malformed Payload (backend validation)
Users' requests may contain missing fields or incorrect data types or values. The backend should validate the inputs and reject those request correctly.

Examples

Frontend Component Validation
Objective: Verify that the UI components correctly validate the user input.
Implementation: We render components (login/registration) using testing libraries, simulate possible user inputs (forms/types), and verify validation behavior (without real APIs); thus, the test remain isolated at the component level.

API Helper Function (Frontend)
Objective: Verify that the frontend API utilities construct and handle the requests correctly from backend without too much lagging
Implementation: We monitor backend responses without calling a live server, and test the formats of requests, success handling and error states. We then verify the successful responses or error messages are correctly shown.

Backend Authorization
Objective: Verify that the protected routes enforce role-based access controls.
Implementation: We simulate authentication tokens and user roles, then test if the endpoints returning the responses correctly. Such as 401 Unauthorized for missing or invalid authentication and 403 Forbidden for insufficient privileges.

Backend Logic with interaction of Database
Objective: Verify if the logic independently of the database, including validating request payloads, transforming input data, returning expected outputs and failure case handling.
Implementation: We call on the database and test if the controller or service functions ensures the correct outputs and the error handling. For example, we simulate correct reads/writes, missing records or database exceptions.

Testing approach - Unit test suite for Data Service

For Inflow Gateway, we use a strong mocking setup for unit tests.

- pytest monkeypatch
- custom test doubles
- fake WebSocket connections
- recording publishers
- fake HTTP responses

On the right are a few corner cases, and how we test for them.

1. **External dependency failures**
   - WebSocket API unavailable
   - Redis unavailable
   - network I/O interruption
2. **Data processing edge cases**
   - incorrect or unexpected data transformation
   - invalid market filtering
   - missing or incomplete payload enrichment
3. **Resilience and recovery scenarios**
   - timeout during data fetch
   - WebSocket connection failure
   - retry behavior after failure
   - graceful degradation when a dependency breaks
4. **Data quality and integrity issues**
   - duplicate message detection
   - malformed or partial payloads
   - incorrect publish behavior

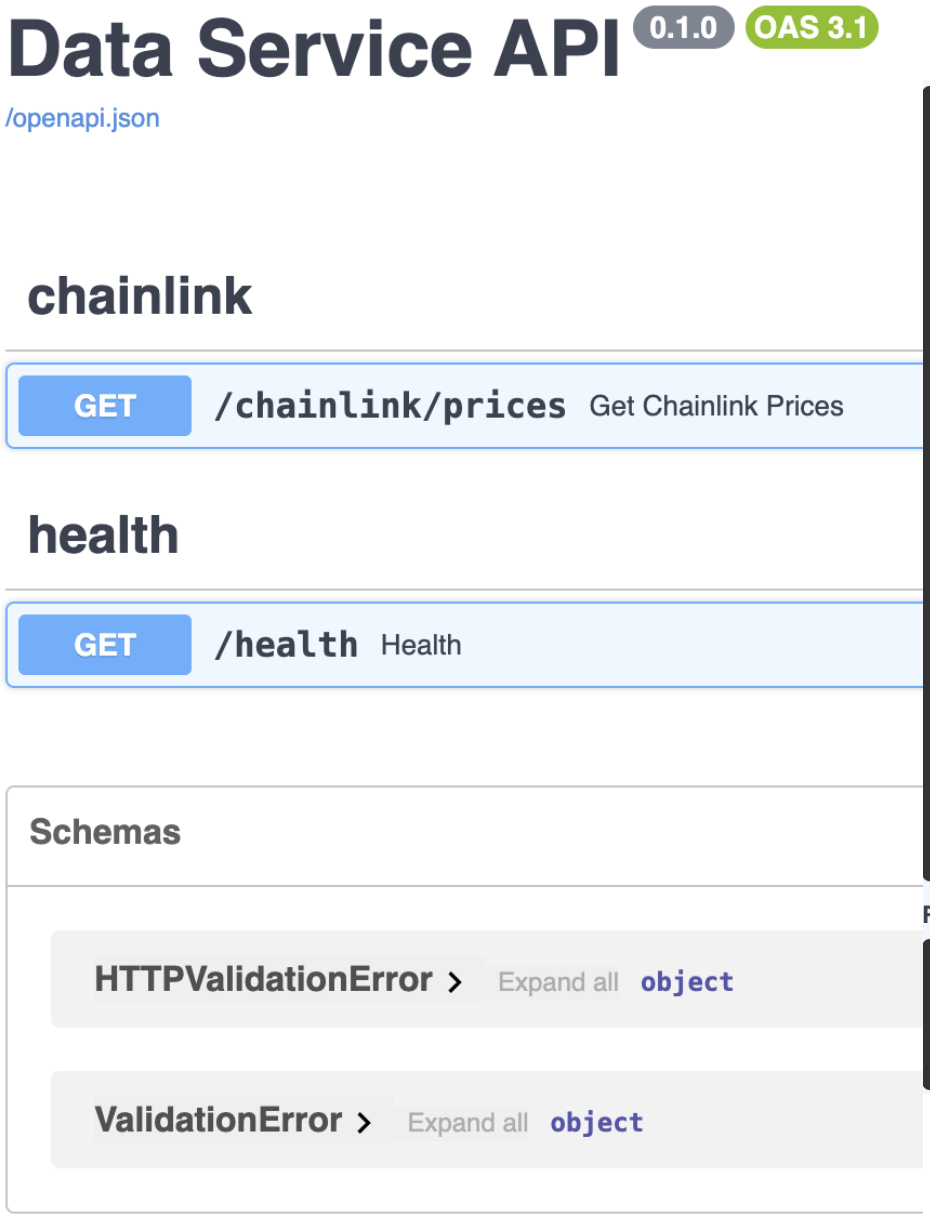
APIs

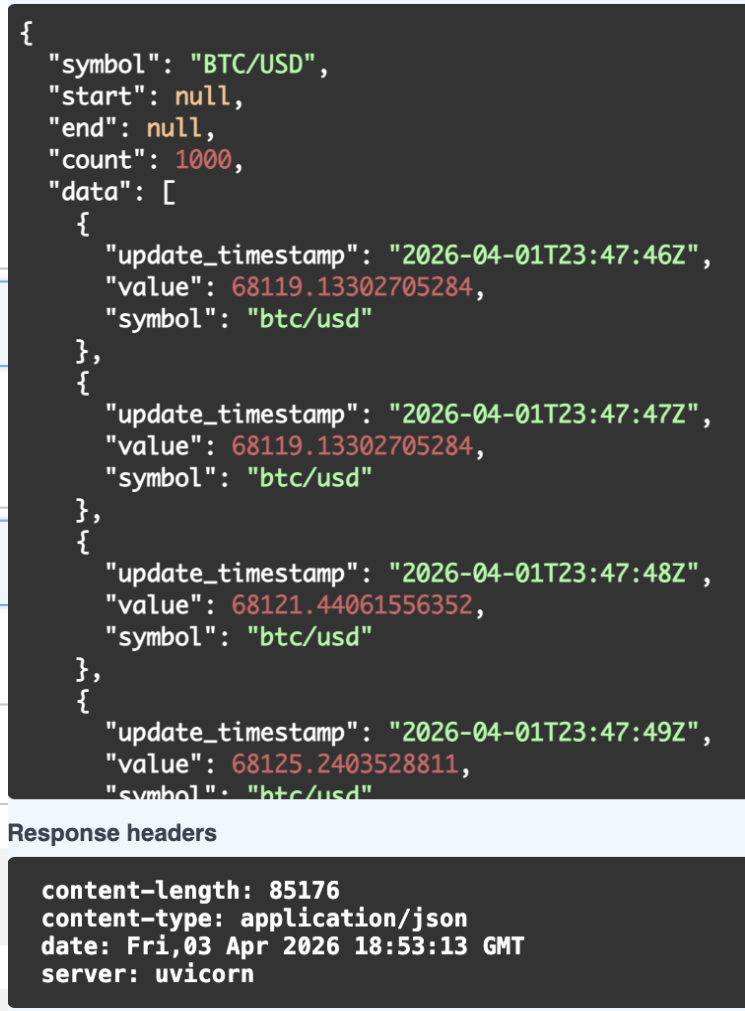
External Polymarket Gamma REST endpoints documented:
- `GET /events`
- `GET /series`
- `GET /trades`

Internal data retrieval endpoint:
These endpoints retrieve data from ClickHouse and expose it for backend consumption
- `POST /internal/clickhouse/write`

Allowed target tables:
- `chainlink_prices`
- `binance_prices`
- `btc_contracts_ob_vol`
- `btc_contracts_ob_midprice`

Please refer to the `api.yaml` file in the repository for full API details.





Error handling

Domain & Logic Safeguards (Task-Driven)

Malformed Ingestion Payload (Schema Changes)
Mechanism: Dead Letter Queue (DLQ). Catch ValidationError, route bad data to a separate Kafka topic, and skip to the next message without crashing the pipeline.

Quantitative Calculation Exceptions (e.g., Missing Data, ZeroDivisionError):
Mechanism: Graceful Degradation. Wrap core math in try-except blocks. Output stale_data flags or safe previous values to prevent NaN or Infinity from polluting the Clickhouse OLAP.

Infrastructure & System Safeguard (NFR-Driven)

Downstream Service Timeout (>500ms API/DB response)
Mechanism: Circuit Breaker Pattern. Temporarily halt requests to the failing service to prevent cascading failures; serve historical fallback data from the Redis cache.

Kafka/WebSocket Connection Drops
Mechanism: Exponential Backoff with Jitter. Catch ConnectionError and reconnect with increasing, randomized delays (e.g., 1s, 2s, 4s + random ms) to avoid network storming.

Spike in Trade Volume (Consumer Lag)
Mechanism: Backpressure & Auto-Scaling. Monitor Kafka consumer lag continuously. If thresholds are breached, trigger AWS ECS to dynamically spin up additional Python processing containers.

Non-Functional Requirements

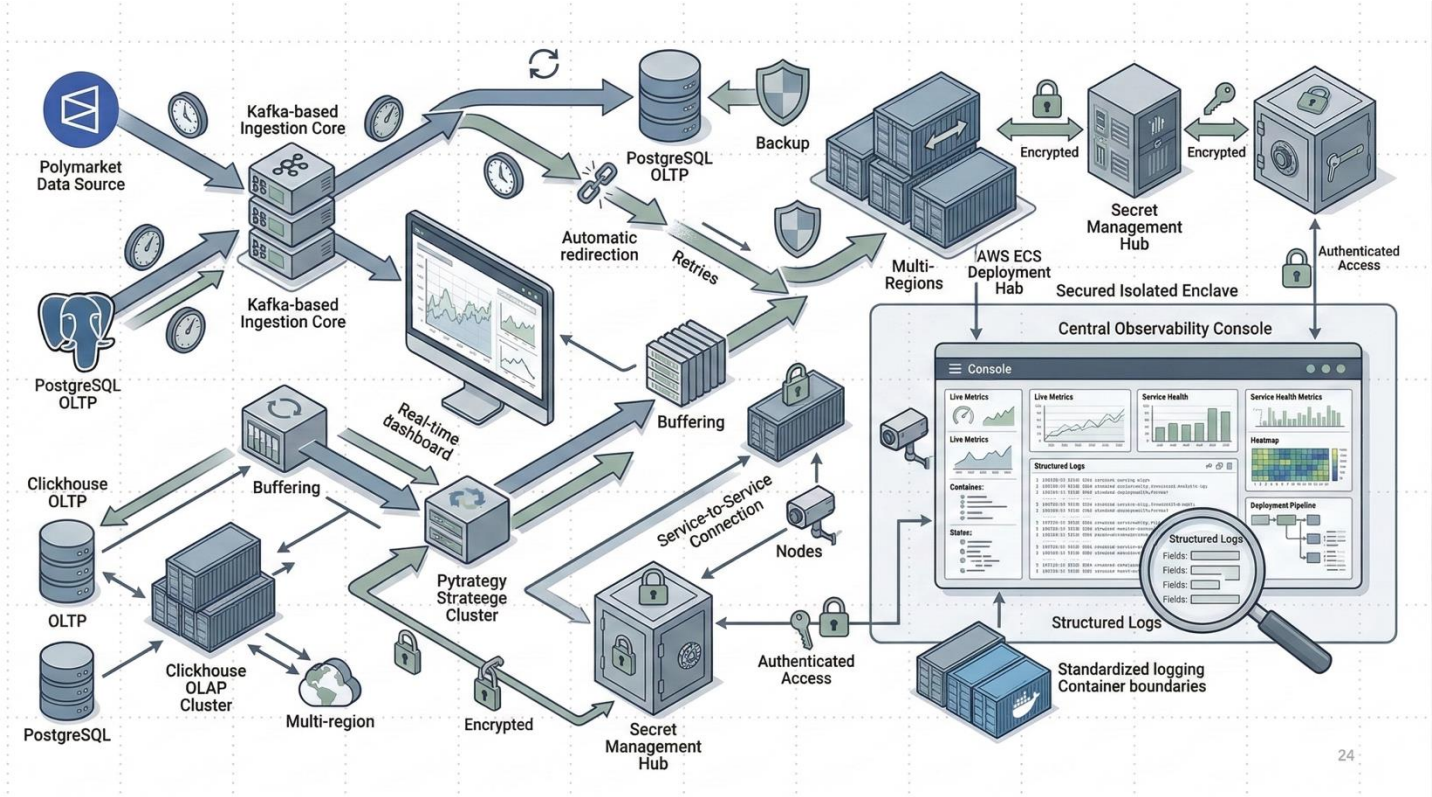
Real-Time Performance
Low-latency ingestion and processing for live Polymarket data, enabling timely analytics and strategy response.

Reliability & Fault Tolerance
Automatic retry, reconnection, buffering,
and graceful degradation to keep the pipeline running under partial failures.

Scalability
Modular services and message-driven architecture allow independent horizontal scaling as market volume grows.

Security & Access Control
Secure secret management, authenticated service access, and environment isolation to protect system resources and credentials.

Maintainability & Observability
Clear module boundaries, containerized deployment, structured logging, and monitoring to simplify debugging, testing, and future extension.

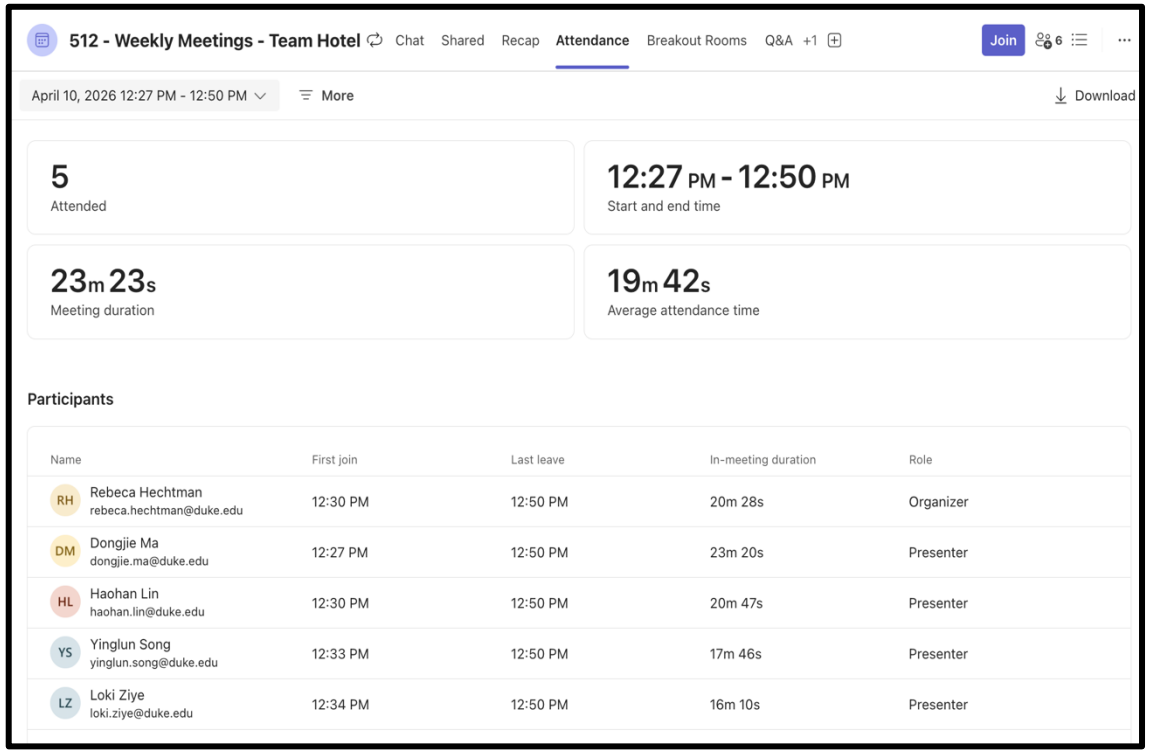


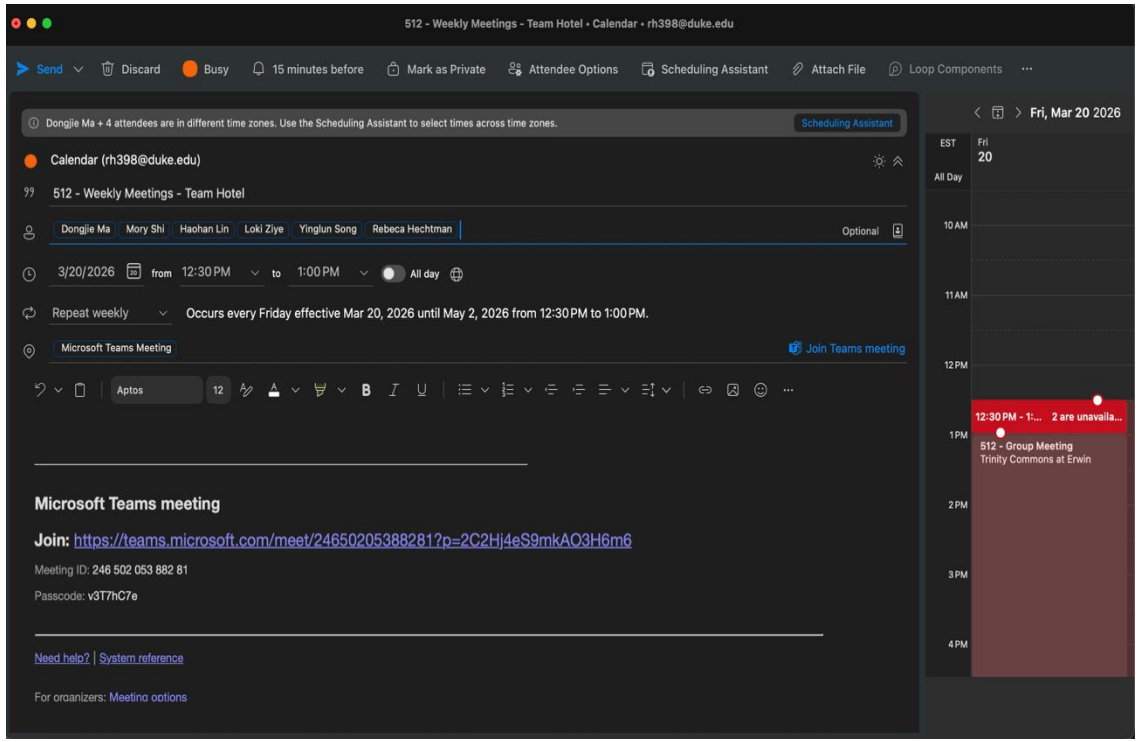
Status, Review and Planning

TA Discussion and Weekly Meetings Scheduled

We successfully met with our assigned TA today (April 10th) to review our project progress and next steps

Next meeting with Dongjie Ma on April 17 (weekly meetings on Friday at 12:30)





Sprint Review Summary

Design & Prototyping: In Week 5, the team continued refining the overall system design by making both user and administrative functionality more detailed and practical. Paper prototypes were expanded further, especially for admin-related features such as user management, password reset, account control, transaction review, and system-wide analysis. The charting component was also improved through additional refinement of interaction design and supporting documentation.

Backend & Integration: The project made clear progress in connecting different parts of the system. The full stack interface was connected to the back-end API, and real API data started replacing earlier placeholder content. On the backend side, the database design was extended to support user roles and permissions, while the data service continued moving forward through transformation updates and testing-related work.

Infrastructure, Testing & Deployment: Week 5 also marked important progress in engineering workflow and deployment readiness. CI/CD work advanced through automated test execution and GitLab SAST integration. Testing efforts were strengthened across both front-end and back-end modules, while containerization work moved forward with Docker file creation and Docker Compose setup. The team also made progress on virtual machine sharing and deployment coordination.

Processes & Collaboration: Team collaboration remained active through continued GitLab issue updates, clearer division of responsibilities, and coordination across modules. Logging and monitoring were also designed and implemented, helping improve maintainability and observability of the system. Overall, Week 5 represented a shift from design and planning into stronger integration, testing, and deployment preparation.

Sprint Retrospective Summary

From Sprint 5, the team did progresses in updating the existing files and applications, including JavaScript, HTML, Docker files etc. And we are implementing our API from Full stack's backend to frontend, including some tests relates to the backend and already finished the API design and implementation from Data services to Backend.

While in technical side, we had docked our applications and did some updates on logging and monitoring (Admin) by the admin's page prototype created last week, and we tried to get the details like column names as well as the structure of the full stack. From Data services side, we had finished the CI/CD design and will try to implement it next week.

Sprint Planning Summary

For Sprint 5, we made some solid progress, and the project has come to its shape. We have implemented most of the features and linked the data server with the full stack. Specifically:

1. Finishing the database and admin logic
2. Focusing on chart constructions
3. Start design and implementing tests

And for next week, sprint 6, we will keep pushing forward and finalizing our project. Key deliverables for the upcoming week include

1. Incorporate the data charts
2. Full test suits
3. Beta version of the application
4. Containerize functions for efficiency

We scheduled our recurring weekly TA meeting for Thursday to ensure we stay ahead with the deadline and have everything delivered in time.

Additional Comments and/or Concerns

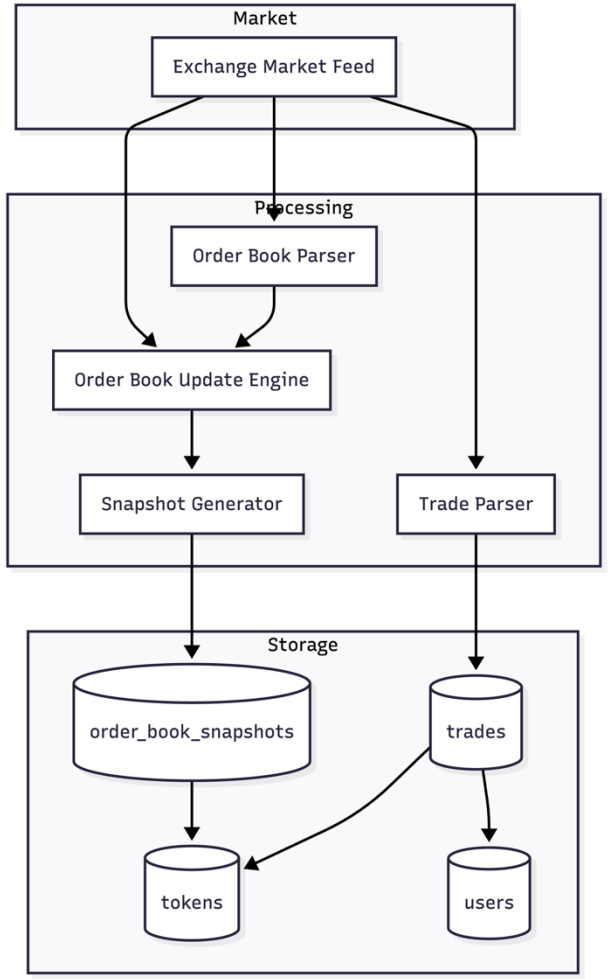
General Comments: We are at the stage of deploying our project to the virtual machine. We need to make sure the CICD flows well, at the same time we are trying to make sure that every week we are doing what we need to do so that we can finish the project on time.

Risks/Blockers: Currently, there are no major blockers. However, we will be actively monitoring concerns as the ones below:

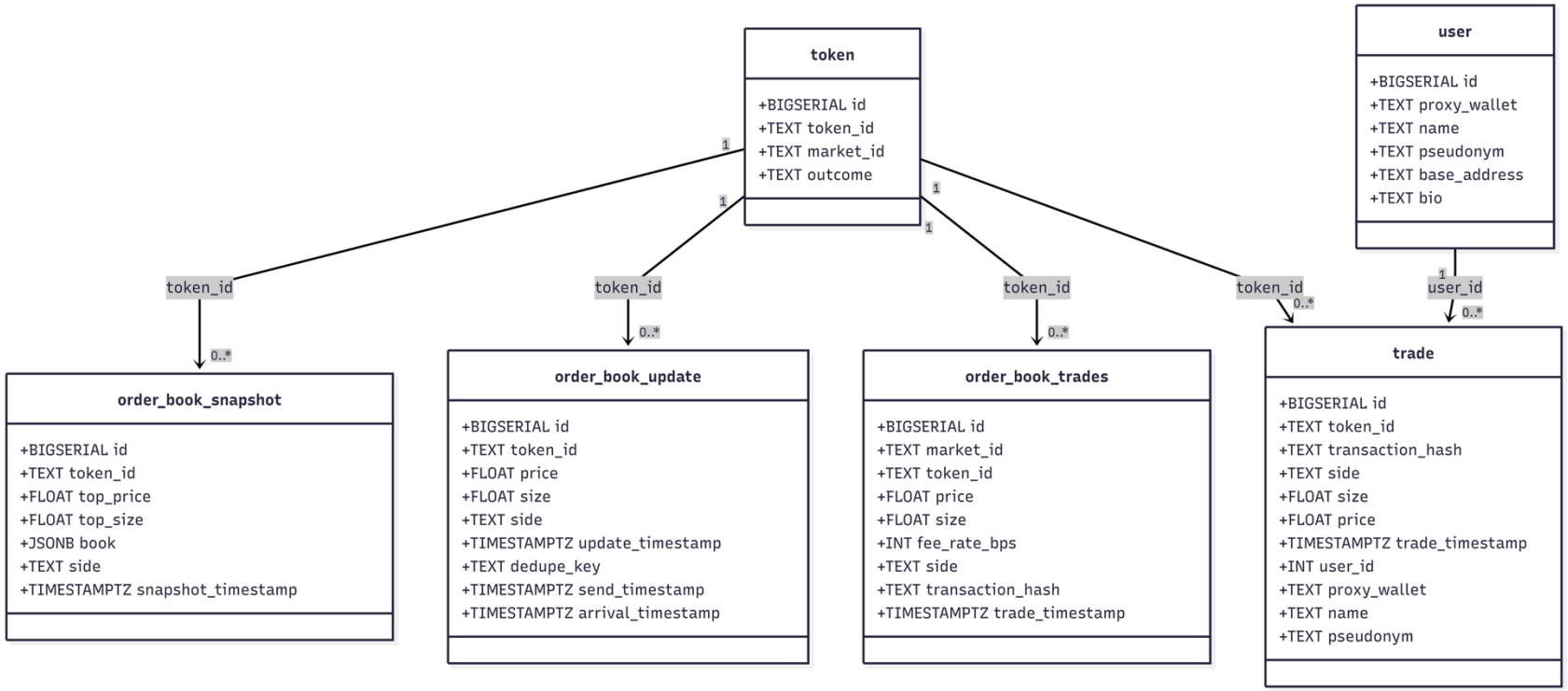
Risk1 – Operational Overhead: Our approach strongly resembles the microservice architecture, while it allows for better modulization, it also adds maintenance overhead to the team. We need to closely monitor the status of each service, and also added more automated orchestrations such as CICD flows to ensure smooth operations.
Risk2 – API Access Concern: Polymarket may retrieve/change the accessibility to their official API endpoints, and it will significantly impede our development.
Risk 3 – Virtual Machine Deployment Needed: We need to deploy the project onto machine so that we know each component works smoothly with each other.
Risk 4 – Massive Data Storage Need: As we are capturing high frequency tick-level orderbook data, the need for storage is overwhelming. We are looking at the need of roughly ~20 GB of storage need per day.

Annex (initial documents created)

UML Class Diagram – Order Book

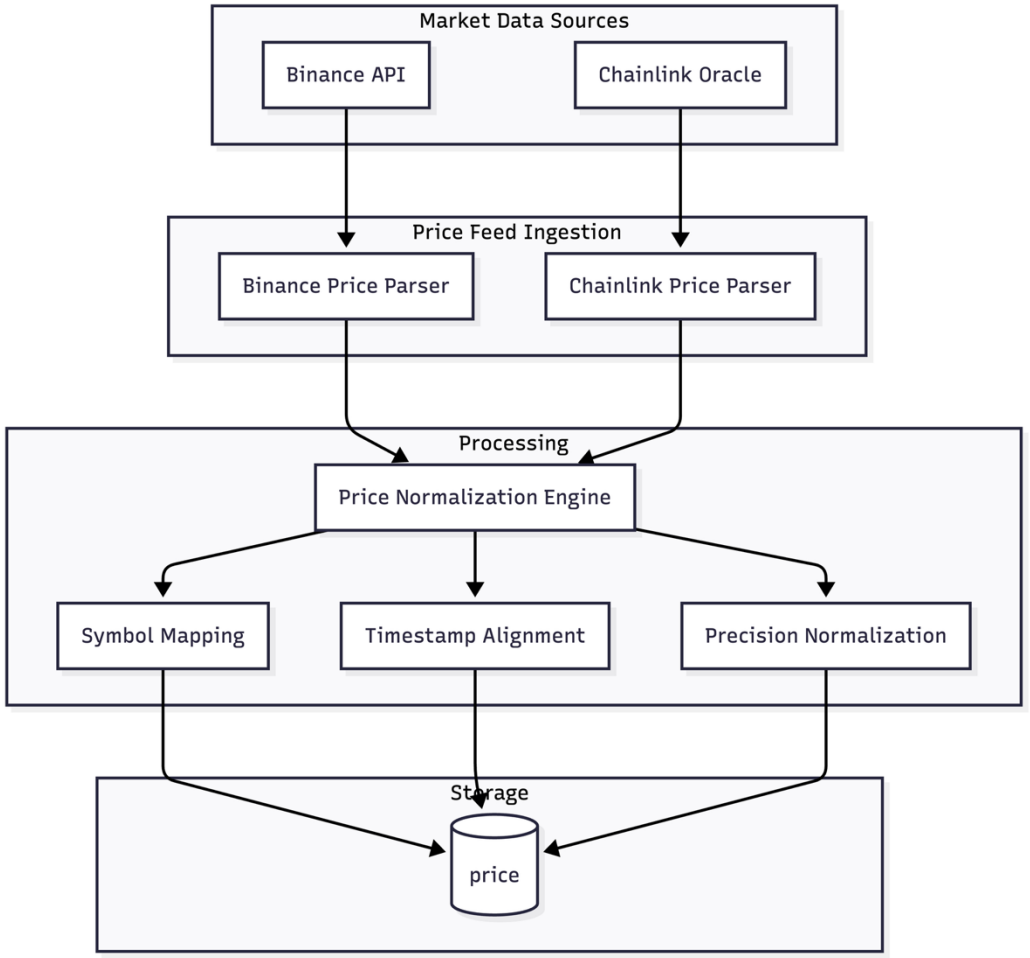


Architecture/pipeline

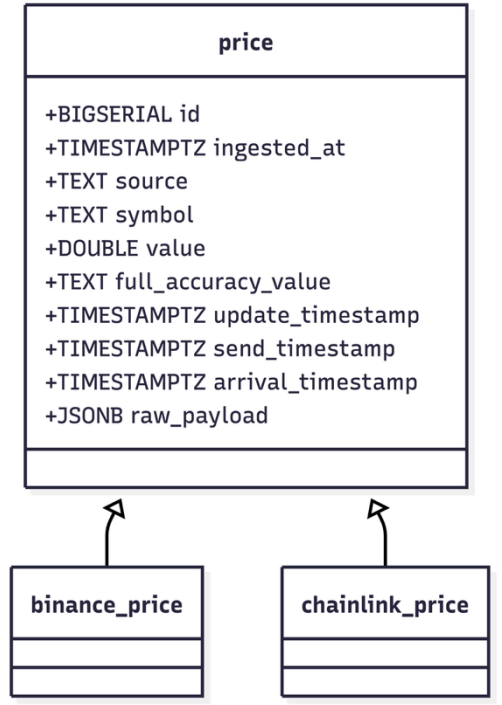


UML Class Diagram

UML Class Diagram - Price

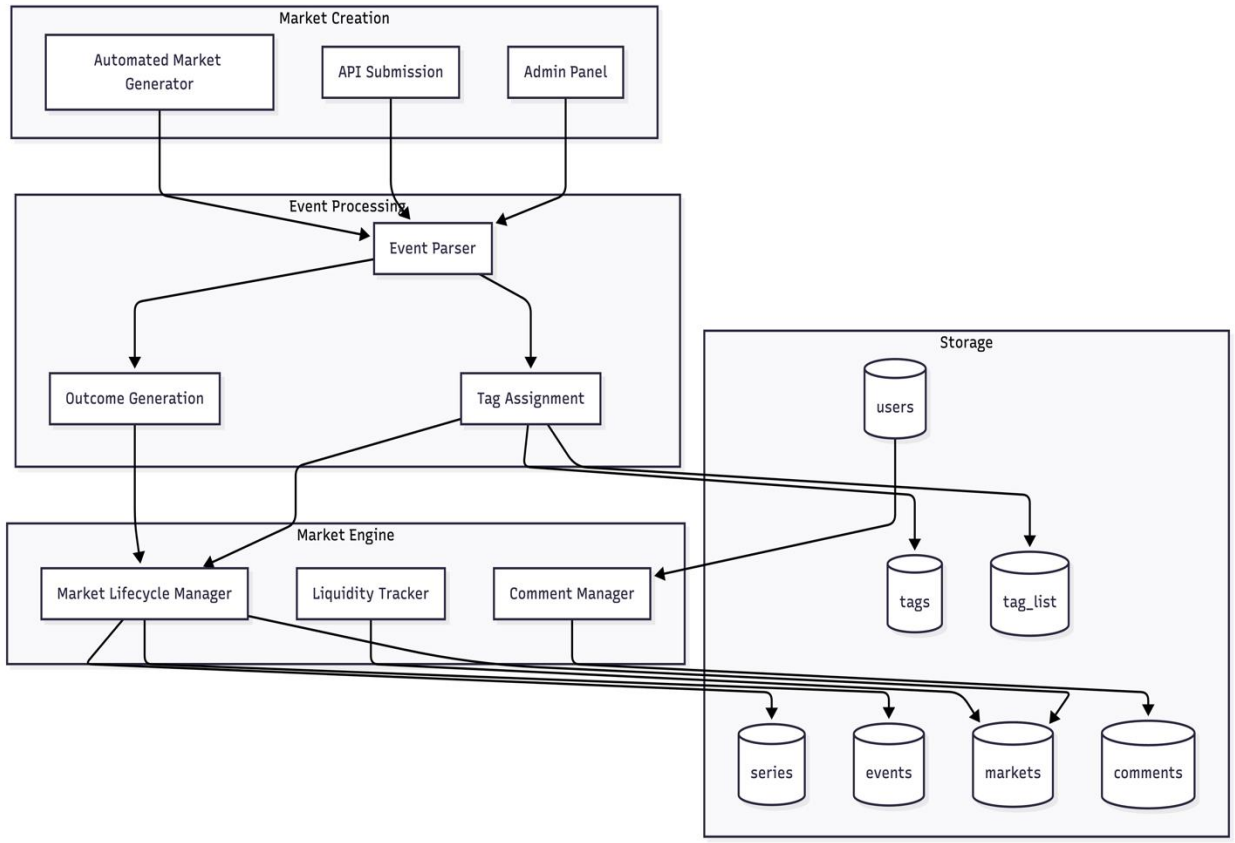


Architecture/pipeline

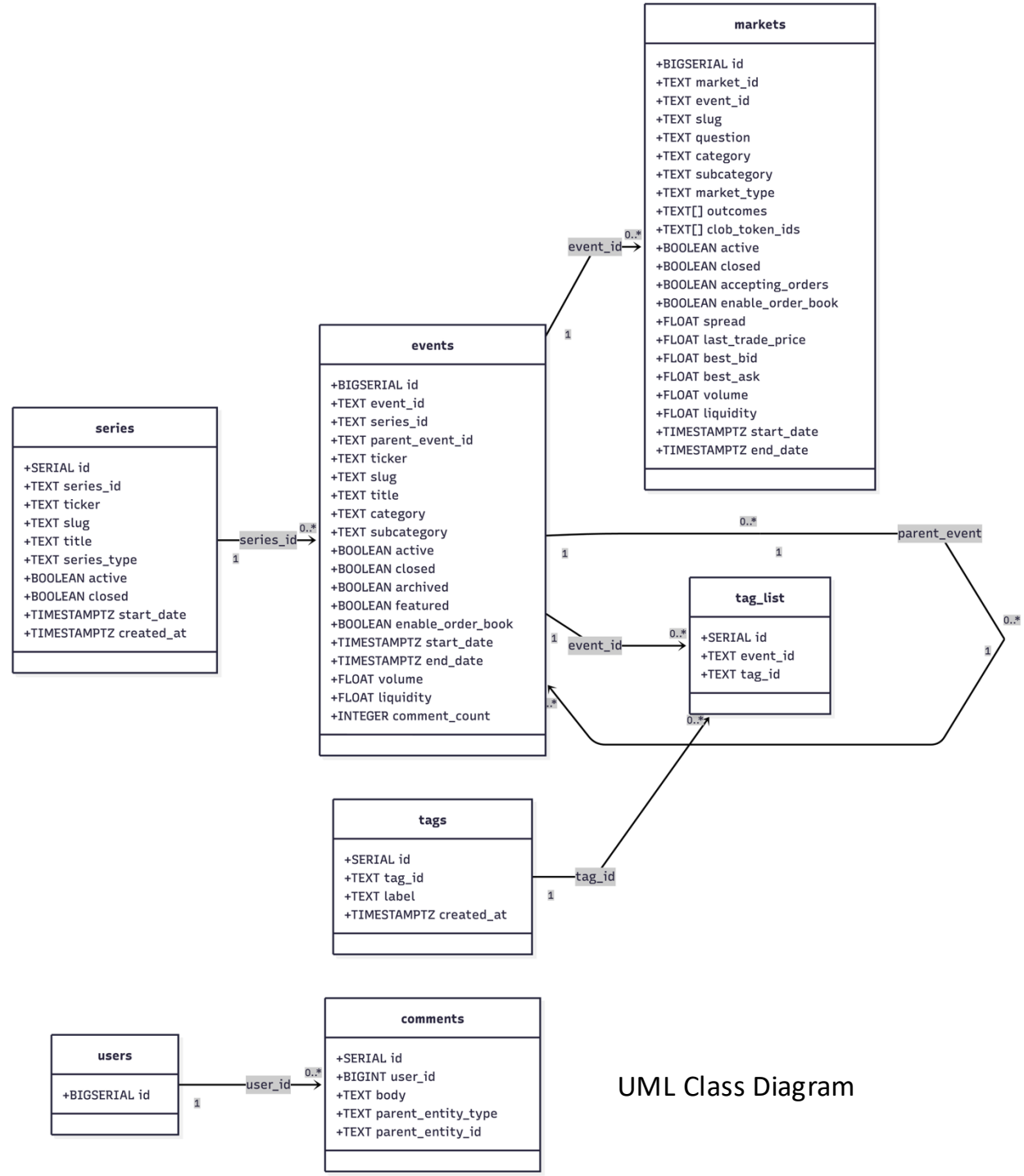


UML Class Diagram

UML Class Diagram - Market



Architecture/pipeline



UML Class Diagram

Class design

Trading/Order Book System

- Core class: token, order_book_snapshot, order_book_update, trade, user
- Relations:
  - Token -> order_book_snapshot
  - Token -> order_book_update
  - Token -> trade
  - User -> trade

Price Oracle System

- Core class: price, binance_price, chainlink_price
- Relations: price --> binance_price / chainlink_price

Market Structure System

- Core class: series, events, markets, tags, tag_list, comments, users
- Relations:
  - series -> events -> markets
  - events -> events (parent_event)
  - events -> tag_list -> tags
  - users -> comments

Usage

Store the order book state and trade executions for each token.

Provides external reference prices from different oracle sources.

Defines the hierarchical structure of prediction markets and their metadata.

Thank you!